# Orbits Without Horizon:

## Convergent Iteration in Shared Geometric Fields as a Model of Open-Ended Computation

Sirsh

March 2026 — Early Draft

**Abstract**

A standard neural network computes in fixed steps: $L$ layers, $L$ passes, done. A brain does not. Cortical representations circulate through the same thalamocortical geometry indefinitely, and the depth of computation is a property of the input and the moment, not an architectural constant.

We develop a theory of **two-phase orbital computation** in autoregressive models built on a shared metric tensor $\mathbf{M}$. The same network, with no parameter changes, operates in two sequential modes: a **compose phase** that iterates $\mathbf{M}$ to build a representation of the reachable state space (maintaining path consistency via the Chapman-Kolmogorov property of the coupling semigroup), and a **predict phase** that collapses this enriched landscape to a point estimate (standard next-token prediction). Composability and prediction pull in opposite directions—our semigroup experiments confirm that CK loss achieves 0.99 composability but hurts prediction—but sequencing them as phases resolves the contradiction: compose builds the map, predict reads it.

This formulation connects the Orbital Response Network to three converging lines of recent work: latent-space reasoning (COCONUT: continuous thoughts encoding multiple futures simultaneously), shared-parameter iteration (Ouro/LoopLM: 2–3× parameter efficiency at 1.4B–2.6B scale), and multi-horizon prediction (Meta: training on multiple future tokens improves single-token inference). The ORN's shared $\mathbf{M}$ provides the mathematical substrate these approaches lack: a coupling geometry whose semigroup structure guarantees that compose iterations build path-consistent representations, not just deeper features.

Preliminary zero-shot experiments confirm the two-phase structure adds value beyond depth: compose-then-predict outperforms double-depth prediction at the same total compute, with correction gates activating more during compose (maintaining diversity) than during predict (allowing convergence). The biological analogy is direct: recurrent cortical processing uses the same circuits for both fast recognition (feedforward sweep) and slow deliberation (recurrent loops), with the mode switch being dynamical, not architectural.

# 1 Motivation: Why Fixed Depth Is a Limitation

## 1.1 The Transformer's Computational Budget

A transformer with $L$ layers performs exactly $L$ steps of computation per token, regardless of difficulty. A function word like "the" receives the same computational depth as a garden-path ambiguity or a long-distance coreference. This is an engineering convenience, not a cognitive principle—and it has formal consequences.

Pérez et al. [4] proved that standard fixed-depth transformers are *not* Turing-complete: they can only recognise languages in a bounded complexity class determined by $L$. Bhatt et al. [6] showed that even bounded-precision RNNs, given growing memory, can simulate a Universal Turing Machine—the key ingredient is not precision but *unbounded iteration*. Dehghani et al. [1] exploited this directly with the Universal Transformer (UT): share all weights, iterate until a

learned halting probability triggers. This makes the architecture Turing-complete [4] and allows variable-depth inference. Pérez et al. further showed that attention itself is Turing-complete when combined with unbounded computation steps [5]—the limitation is not in the attention mechanism but in the fixed depth.

UT shares *everything*—attention coupling, response heads, and the FFN—which our prior work shows is too aggressive. Response heads benefit from per-layer specialisation (sharing costs ∼4% quality in controlled experiments), and the blanket sharing prevents architectural rebalancing.

### 1.2 What We Learned from Fixed-Depth SharedM

Our companion paper (*Stigmergic Residue*) established that sharing only the coupling geometry $\mathbf{M} = AB^\top$ across layers, while keeping response heads and FFNs per-layer, achieves transformer parity with 5× fewer coupling parameters. The shared $\mathbf{M}$ defines a field geometry; tokens couple based on their position in this geometry rather than through per-layer learned projections.

A natural question: if $\mathbf{M}$ defines a geometry, can tokens iterate in this geometry beyond the fixed training depth? Could inference at $T$=12 or $T$=100 yield better representations than $T$=6?

### 1.3 Experiment 10: The Answer Is No (Without the Right Training)

We trained SharedM at fixed $T$=6 on character-level Shakespeare and evaluated at $T \in \{1, 2, \ldots, 24\}$. The result:

| $T$ | Val Loss | |
|---|---|---|
| 1 | 3.70 | underfitting |
| 4 | 2.93 | improving |
| **6** | **2.25** | **training depth (optimum)** |
| 8 | 2.43 | diverging |
| 12 | 2.98 | worse than $T$=3 |
| 24 | 4.10 | near random |

Loss degrades monotonically beyond $T$=6. The orbit does not converge to a useful fixed point.

However, the KL divergence between consecutive iterations *does* decrease (from ∼15 to ∼0.3)—the predictions stabilise, but to a degenerate distribution. The geometry is doing *something* convergent; it is just not converging to the right place.

*Remark.* This is exactly what one expects from a system trained at fixed depth. Gradient signals never reached iterations 7–24, so there is no pressure for those iterations to produce useful representations. The response heads are calibrated for exactly $T$=6 steps of residual stream evolution. Push further and the stream enters a region the heads cannot decode—not because the geometry is wrong, but because the decoder was never trained for that region.

## 2 Theory: Convergent Orbits in Shared Geometries

### 2.1 The Fixed-Point Formulation

**Definition 1** (Orbital fixed point)**.** Given a shared block $\Phi$ (coupling via $\mathbf{M}$, response via $W_V, W_O$, transformation via FFN) and an initial embedding $\mathbf{x}_0 = \text{Embed}(w)$, define the orbit:

$$\mathbf{x}_{t+1} = \Phi(\mathbf{x}_t)$$

A **fixed point** is a state $\mathbf{x}^*$ such that $\Phi(\mathbf{x}^*) = \mathbf{x}^*$. Equivalently, the residual update $\Delta_t = \Phi(\mathbf{x}_t) - \mathbf{x}_t$ vanishes.

In a residual network, $\Phi(\mathbf{x}) = \mathbf{x} + f(\mathbf{x})$, so a fixed point requires $f(\mathbf{x}^*) = 0$: the attention and FFN contribute nothing. This is the state where the representation is "done"—further iteration would not change it.

### 2.2 Conditions for Convergence

**Theorem 1** (Banach fixed-point, applied)**.** *If $\Phi$ is a contraction mapping on a complete metric space—i.e., $\|\Phi(\mathbf{x}) - \Phi(\mathbf{y})\| \leq \gamma\|\mathbf{x} - \mathbf{y}\|$ for some $\gamma < 1$—then the orbit $\mathbf{x}_t$ converges to a unique fixed point $\mathbf{x}^*$ from any initialisation.*

For our architecture, $\Phi$ includes LayerNorm, softmax attention, GELU activations, and residual connections. It is *not* globally contractive—softmax is not Lipschitz-bounded in general, and the residual connection preserves norm. But it may be *locally* contractive near attractors, which suffices for practical convergence.

Deep Equilibrium Models [2] showed that this works in practice: a single weight-tied layer iterated to convergence achieves competitive performance with constant memory, using implicit differentiation to backpropagate through the fixed point.

### 2.3 The Biological Analogy

Cortical computation is recurrent. A visual scene is not processed in one feedforward pass; representations circulate through the thalamocortical loop, with each pass refining the interpretation [3]. The "depth" of processing is not fixed—it depends on the stimulus, the task, and the available time.

- **Fast responses** (reflexive, automatic) use few iterations. The orbit barely leaves the initial embedding.
- **Slow responses** (deliberative, complex) use many iterations. The orbit traverses a longer trajectory before converging.
- **Rumination** is an orbit that fails to converge—the representation keeps cycling without settling.

The shared geometry (the cortical connectivity, the synaptic weight matrix) is the same in all cases. What varies is how long the state is allowed to evolve. This is precisely the variable-$T$ regime we propose.

### 2.4 Why This Differs from Universal Transformer

UT achieves variable depth through ACT: a learned scalar halting probability per token per iteration. The mechanism is a *gate*—the model explicitly decides when to stop. This requires training the halting predictor alongside the computation, adding a regularisation term to avoid trivial solutions ($T$=1 always).

We propose a different mechanism: **convergence-based halting**. The iteration stops when $\|\mathbf{x}_{t+1} - \mathbf{x}_t\| < \theta$—when the orbit has converged. No learned halting gate, no additional parameters, no regularisation. The halting criterion is a *consequence* of the geometry, not a separate learned decision.

This requires the geometry to be contractive, which requires training with variable depth. The training signal is: "your representation should be useful at *any* iteration, and it should stop changing when it is done."

**Turing completeness.** The formal results [4, 5] establish that unbounded iteration over shared weights is *sufficient* for Turing completeness. UT achieves this; our architecture, with convergence-based halting and shared $\mathbf{M}$, achieves it in principle for the same reason—iteration is unbounded, and the shared weights are a finite description of the transition function. But Turing completeness is not our goal. We are interested in a narrower and more practical question:

does variable-depth iteration *improve language modelling quality* by allocating more computation to difficult tokens? The Turing completeness results tell us the architecture is not fundamentally limited; the experiments will tell us whether the geometry learns to use the extra computation productively.

# 3 Proposed Experiments

## 3.1 Exp 10b: Variable-Depth Training

Train SharedM with $T$ sampled uniformly from $[2, T_{\max}]$ per batch. For each sample, the model must produce useful predictions at the sampled depth. This forces the geometry to be useful at any iteration count, not just the training depth.

**Variants.**

1. **Uniform** $T$: sample $T \sim \text{Uniform}(2, 16)$ per batch.
2. **Curriculum**: start with small $T$ (1–4), gradually increase $T_{\max}$ during training.
3. **Multi-$T$ loss**: compute loss at *every* iteration, weighted by $w_t = t^{-1}$ (early iterations matter more for fast responses, later iterations for difficult inputs).
4. **DEQ-style**: use implicit differentiation to find the fixed point; backpropagate through the fixed-point condition rather than through the iteration.

**Metrics.**

- Val loss at each $T$ (should plateau rather than diverge).
- Convergence rate: $\|\mathbf{x}_{t+1} - \mathbf{x}_t\|$ as a function of $t$ (should decay).
- Per-token iteration count: how many iterations before $\Delta < \theta$? Prediction: rare/ambiguous tokens need more iterations.
- Quality vs. compute: Pareto frontier of loss vs. average $T$ at inference.

## 3.2 Exp 10c: Difficulty-Adaptive Inference

Given a model trained with variable $T$, run inference with convergence-based halting ($\|\Delta\| < \theta$). Measure:

- Which tokens halt early? (Prediction: function words, predictable continuations.)
- Which tokens halt late? (Prediction: garden-path disambiguation, rare words, long-distance dependencies.)
- Does average iteration count correlate with human-judged difficulty?

## 3.3 Exp 10d: Open-Ended Generation

The most speculative test: generate text with no iteration limit, halting only when the representation converges. Compare output quality (measured by human evaluation or a larger model's perplexity) against fixed-depth generation. If variable-depth generation produces more coherent text on difficult passages, the "thinking longer" hypothesis is supported.

# 4 Two Kinds of Orbit: T-Loops and R-Loops

## 4.1 The Distinction

There are two distinct iteration structures in autoregressive generation with shared geometry:

**$T$-loops (inner iteration).** Within a single forward pass, the residual stream iterates through the shared geometry $T$ times: $\mathbf{x}_0 \to \mathbf{x}_1 \to \cdots \to \mathbf{x}_T$. Each step applies the same $\mathbf{M}$ to

an evolving representation of the *same input*. This is "thinking longer about one thing." Experiment 10 tested this and found divergence without variable-depth training.

**$R$-loops (outer iteration).** The model generates a token, appends it to the sequence, and runs a new forward pass on the extended input. Each $R$-step is a complete forward pass (with $T$ inner steps), and the output of one pass becomes part of the input to the next. This is autoregressive generation—"continuing the conversation."

$T$-loops iterate in *representation space*: the same tokens, refined through the same geometry. $R$-loops iterate in *sequence space*: the sequence grows, new tokens enter the field, and the representation is recomputed. Every language model already performs $R$-loops indefinitely. The variable-depth question is about $T$-loops: can the inner iteration be open-ended?

### 4.2 The Field as Memory

In a standard transformer, the KV cache stores per-layer key and value projections for all previous tokens: $L \times N \times d$ entries. Each layer maintains its own cached projections—a lookup table of precomputed results, one per depth.

In ORN, there is only one $\mathbf{M}$. A token's coupling to other tokens is determined by where they sit in $\mathbf{M}$'s geometry *at that moment*, not by stored per-layer projections. The "cache" is the residual stream states of all previous tokens after $T$ iterations—the field itself. This is $N \times d$, not $2L \times N \times d$: a factor of $2L$ smaller. A standard KV cache stores keys *and* values at *every* layer; the ORN field stores only the residual state, once. At $L$=13 (the SharedM configuration from the companion paper), this is a 26× reduction in context memory.

But the difference is not just quantitative. In the standard transformer, the KV cache is an implementation optimisation—a way to avoid recomputing projections. In ORN, the residual stream *is* the computational substrate. When a new token enters, it reads the field (the states of all previous tokens) through $\mathbf{M}$, and contributes back to the field. The "cache" is not a cache—it is the environment.

This is where the stigmergic analogy becomes precise rather than metaphorical. The field (residual states) persists. New agents (tokens) read it and deposit into it. The coupling rule ($\mathbf{M}$) is the same for everyone. The pheromone trail is not a cache of ant decisions; it *is* the medium through which ants coordinate. The residual stream is not a cache of attention results; it *is* the medium through which tokens coordinate.

### 4.3 Composition of T and R

The two loops compose. Each $R$-step generates a token using $T$ inner iterations. If $T$ is variable, hard tokens get more inner iterations, producing better predictions that create better context for subsequent $R$-steps.

A more speculative possibility: the $T$ and $R$ loops need not be strictly nested. A model could iterate, producing intermediate states that are themselves meaningful—like a draft progressively refined. Each inner iteration both refines the current representation *and* implicitly prepares the field for the next token. In the limit, the boundary between "thinking" (inner $T$-loop) and "generating" (outer $R$-loop) dissolves: the orbit through the shared geometry is simultaneously a refinement of the current prediction and an evolution of the field state.

This corresponds to the biological observation that thinking and acting are not strictly sequential. Speech production, for instance, involves simultaneous planning of upcoming words while articulating current ones—the cortical "field" is processing multiple timescales at once through the same geometry.

# 5 Context as Field Capacity, Not Memory

## 5.1 The Context Window Problem, Reframed

In a standard transformer, the context window is a *memory* question: how many KV pairs can we store? Each previous token occupies $L \times d$ entries in the cache—its per-layer key and value projections, each one a specific lookup address. A 128K context window means $128{,}000 \times L \times d$ entries sitting in memory. The model can directly address any of them through the attention mechanism. Context is storage.

In ORN, context is *field state*—the configuration of the residual stream across all previous tokens. The "context window" is not a memory limit but a **field capacity** question: how many distinct token states can the field support before $\mathbf{M}$'s geometry can no longer distinguish them?

This is related to the effective rank of $\mathbf{M}$. If $\mathbf{M}$ has effective rank $r$, the field geometry supports $\sim r$ distinguishable coupling dimensions—$r$ independent "channels" through which tokens can influence each other. Two tokens that project to the same region of $\mathbf{M}$'s eigenspace are indistinguishable from the coupling perspective, regardless of how different their content is. The context capacity is set by the *richness of the geometry*, not by the size of a cache.

## 5.2 The Drift Problem in Long Orbits

This reframing reveals why open-ended $T$-iteration is hard. $\mathbf{M}$ is a single $d \times d$ matrix. It defines one geometry. That one geometry must support *every possible context*—a conversation about physics, a story about a dog, a code snippet. At $T$=6 this works: the response heads and FFNs read the field state and extract context-appropriate information. The field carries the context; $\mathbf{M}$ provides the coupling rule; the response heads do the context-dependent extraction.

But as $T$ grows, the field state keeps evolving under the same $\mathbf{M}$. Without context-dependent correction, representations drift toward $\mathbf{M}$'s dominant attractors *regardless of the original prompt.* This is exactly what Experiment 10 showed: the orbit converges to a degenerate attractor that has lost the contextual information. The structural geometry dominates; the specific details wash out.

The problem is not that $\mathbf{M}$ is too simple—it is that $\mathbf{M}$ captures what is *context-invariant* (structural coupling rules), and long iteration amplifies the invariant at the expense of the particular. "Verbs attend to subjects" is the same in every conversation; after enough iterations under this rule, the field forgets *which* verb and *which* subject.

## 5.3 The Ontological Pathway as Contextual Anchor

This is where the ontological pathway—introduced in the companion paper as a tiny identity-routing mechanism—becomes essential rather than optional.

**$\mathbf{M}$ (orbital pathway)** captures structural coupling: "what kinds of things attend to what kinds of things." This is context-independent. It is the same in every conversation.

**The ontological pathway** carries context-specific bindings: "she = Lily," "the toy = the red ball from paragraph 2." These are the details that distinguish one orbit from another given the same geometry.

For open-ended $T$-iteration, the ontological pathway must *actively re-inject* contextual specifics at each iteration—not as a static per-token correction (which washes out under repeated application of $\mathbf{M}$) but through its own attention mechanism that reads the current field state and re-grounds the representation in the specific context.

Each $T$-step, $\mathbf{M}$ evolves the structural representation (the orbit advances). Each $T$-step, the ontological pathway re-anchors it in the particular context (the orbit stays on the right trajectory). Without the anchor, long orbits lose specificity—the structural geometry pulls all contexts toward the same attractors. With it, the orbit can continue indefinitely because the contextual bindings are refreshed at every step.

**Conjecture 1** (Ontological anchoring enables convergent orbits)**.** *A model with shared* $\mathbf{M}$ *alone will diverge or collapse under extended iteration (Experiment 10 confirms this). A model with shared* $\mathbf{M}$ *plus an active ontological pathway—refreshing context-specific bindings at each iteration—can support convergent orbits of arbitrary length, because the ontological corrections prevent drift toward degenerate attractors.*

This is directly testable: run Experiment 10 with the dual-pathway architecture from Experiment 4d, and compare convergence behaviour. Prediction: the dual-pathway model will show slower divergence or outright convergence beyond the training depth, because the ontological pathway anchors the orbit.

### 5.4 The Biological Parallel

The cortex–hippocampus division maps precisely onto this analysis. The cortex ($\mathbf{M}$) provides general-purpose dynamics—structural coupling rules that are the same across all contexts. The hippocampus (ontological pathway) provides episodic bindings that keep the current train of thought grounded in *this specific* context.

Rumination—when thinking goes in circles without resolving—may be what happens when the ontological grounding is too weak to steer the orbital dynamics. The structural geometry cycles through its attractors, but without strong enough contextual anchoring, the orbit never settles on a resolution. In this framing, the "ontological pathway" is not just a parameter-efficiency trick; it is the mechanism that makes open-ended computation *useful* rather than merely open-ended.

## 6 New Evidence from the Semigroup Analysis

Since the initial draft of this paper, two experimental programmes have produced results that directly inform the convergent orbit question: the variable-depth training experiments (Exp 10b) and the semigroup deep dive on the 50M SharedM checkpoint.

### 6.1 Exp 10b: Variable-Depth Training Works

We ran all four variants proposed in Section 3.1 on character-level Shakespeare ($d$=64, $L$=6, 3,000 steps). All four converge—the orbit stabilises rather than diverging:

| **Strategy** | **Best $T$** | **Val Loss** | **Final KL** | **Converges?** |
|---|---|---|---|---|
| Fixed-$T$=6 (baseline) | 7 | 3.36 | 0.015 | Yes |
| Uniform-$T$ | 6 | **3.37** | 0.012 | Yes |
| Multi-$T$ loss | 7 | 3.40 | 0.009 | Yes |
| Curriculum | 4 | 3.39 | 0.008 | Yes |

Table 1: Variable-depth training results. All strategies produce convergent orbits. KL divergence between consecutive iterations drops to ∼0.01 at depth 12, vs ∼0.3 for the fixed-depth model in Experiment 10. Curriculum and Multi-$T$ achieve the lowest final KL.

The key finding: **variable-depth training eliminates the divergence problem**. The fixed-depth model from Experiment 10 diverged catastrophically beyond $T$=6 (loss 4.10 at $T$=24). The variable-depth models maintain stable loss across all tested depths, with KL convergence 10× better than the fixed-depth baseline.

The best val loss at training depth is comparable across all strategies (3.36–3.40), so variable-depth training does not sacrifice fixed-depth quality to gain convergence. This resolves the main open question from Section 1.3: the geometry *can* be trained to converge, without a learned halting gate, using only variable $T$ during training.

### 6.2 The Homogenisation Mechanism

The semigroup analysis (companion document: *The Coupling Semigroup in Practice*) measured field state evolution across 9 layers in the 50M SharedM model. The finding directly explains both why convergent orbits work and why they have limits:

**Consensus formation.** Mean pairwise token cosine rises from 0.21 (embedding layer) to 0.91 (layer 9). Tokens converge toward a shared representation as they absorb relational context through $\mathbf{M}$. This is not signal loss—it is consensus. Each iteration, the response heads write back what they learned from $\mathbf{M}$'s coupling pattern, building a shared representation of the relational structure.

**Selective preservation in M's coupling subspace.** Per-token variance projected onto $\mathbf{M}$'s top-10 singular vectors grows 1.8–3.4× faster than overall variance. The response heads preferentially amplify signal in the directions that $\mathbf{M}$ reads. This is the mechanism that makes convergent orbits possible: even as the overall field homogenises, the coupling-relevant contrast is maintained.

**The decay sets a depth horizon.** The selective preservation ratio decays: 3.4× at layer 1, 1.8× at layer 9. The response heads maintain coupling-relevant contrast, but not indefinitely—the homogenisation encroaches on the coupling subspace too, just more slowly. This explains why fixed-depth models diverge beyond training depth (the coupling signal eventually degrades below usefulness) and predicts that even variable-depth models will have a practical depth limit.

### 6.3 K-Vector Phase Transitions Support Variable Depth

The semigroup analysis found that the key vector for "Lily" undergoes a phase transition at layer 4: layers 0–3 treat it as a name token, layers 4–8 treat it as a narrative agent (K-space cosine between the two groups: $-0.22$). This functional role switch is driven entirely by the field state, not by layer-specific parameters.

This directly supports the variable-depth proposal: different tokens need different amounts of processing. A function word like "the" may converge quickly (it has a simple, consistent role). A proper noun like "Lily" needs at least 4 layers to complete its role transition. An ambiguous pronoun may need even more. Variable-$T$ inference would naturally allocate more iterations to tokens with slower-converging orbits.

### 6.4 Coreference and the Ontological Anchoring Conjecture

The semigroup analysis showed that coreference ("Lily" = "She") emerges in K-space without supervision: co-referent tokens develop more similar key representations than non-co-referent tokens at deeper layers.

This is exactly the kind of signal that extended iteration threatens to destroy. The coreference signal lives in per-token contrast within $\mathbf{M}$'s coupling subspace—the same contrast that the selective preservation measurement shows decaying from 3.4× to 1.8× over 9 layers. Extended iteration beyond 9 layers would push this ratio below the threshold needed for coreference resolution.

This strengthens the ontological anchoring conjecture (Section 4.3): the ontological pathway's role is to **re-inject per-entity contrast at each iteration**, counteracting the homogenisation that $\mathbf{M}$ naturally produces. We can now be specific about what must be preserved: it is the per-token variance in $\mathbf{M}$'s top singular vectors. The ontological pathway must maintain this variance above the coupling threshold, even as the overall field converges.

The measurement also gives us a diagnostic: if the ontological pathway is working, the selective preservation ratio should *not* decay with depth—it should stabilise or even grow. The ratio $\text{Var}_{\text{projected}}/\text{Var}_{\text{overall}}$ is a direct measurement of whether the entity-level contrast is being maintained where it matters.

### 6.5 Variable Depth at 50M: The Full Picture

We ran the 50M SharedM checkpoint (trained at $T$=9) at depths $T \in \{1, \ldots, 18\}$ on TinyStories. This is the Experiment 10 analysis at a scale where $\mathbf{M}$ has genuine spectral richness (effective rank 51, condition number 700,000).

| $T$ | Val Loss | vs $T$=9 | KL vs prev | Residual norm |
|---|---|---|---|---|
| 1 | 6.004 | | — | 6.9 |
| 5 | 3.398 | | 0.246 | 9.9 |
| 8 | 2.209 | | 0.171 | 13.5 |
| **9** | **2.145** | **training depth** | 0.198 | 16.7 |
| 10 | 2.225 | +3.7% | 0.182 | 19.5 |
| 11 | 2.349 | +9.5% | 0.077 | 20.4 |
| 12 | 2.514 | +17.2% | 0.075 | 20.7 |
| 14 | 2.975 | +38.7% | 0.080 | 20.6 |
| 16 | 3.253 | +51.7% | 0.057 | 21.2 |
| 18 | 3.597 | +67.7% | 0.115 | 24.2 |

Table 2: Variable depth inference on SharedM-50M. The model degrades beyond training depth, but more gracefully than the Shakespeare model from Experiment 10 (+3.7% at $T$+1 vs +8%).

Two findings update the picture from Experiment 10:

**1. Graceful degradation near training depth.** $T$=10 is only 3.7% worse than $T$=9—the model has approximately one layer of "slack" where the geometry still produces useful coupling. For comparison, the Shakespeare model (Exp 10, $d$=64) degraded by 8% at $T$+2. The 50M model degrades half as fast initially, consistent with the richer $\mathbf{M}$ geometry (effective rank 51 vs much lower for the toy model) providing more room before per-token contrast drops below the coupling threshold.

**2. KL drops then plateaus; loss keeps rising.** The KL divergence between consecutive layers drops from ∼0.20 to ∼0.06 by $T$=16—the predictions stabilise, each additional layer making smaller changes. But they stabilise to the wrong distribution (loss rises from 2.15 to 3.60). The residual norm plateaus at ∼20–21 for $T$=11–16, then jumps at $T$=18. The orbit is converging to a degenerate attractor, exactly as predicted in Section 4.2.

This confirms the core thesis of this paper: **the geometry supports convergence (KL drops), but not to a useful fixed point (loss rises)**. The fix must come from training: variable-$T$ training (Exp 10b, confirmed at small scale) or ontological anchoring (conjectured, supported by the selective preservation analysis showing coupling-subspace contrast decaying from 3.4× to 1.8× over 9 layers).

The practical implication: a 50M ORN model can safely run at $T$=10 (one extra layer beyond training) with only 3.7% quality loss. This is a modest but real form of adaptive compute—easy tokens could use $T$=7 or $T$=8 while hard tokens get $T$=10—even without variable-depth training.

### 6.6 Multi-Horizon Training: A Stronger Path to Adaptive Compute

A complementary approach to variable-$T$ training has been tested on TinyStories (companion technical report: *Path Integrals and the ORN*). Instead of training with variable depth on next-token prediction, we train depth $L$ to predict the token $L$ steps ahead—the "path integral" training objective.

Three models on the same SharedM architecture ($d$=64, $L$=4):

| Model | Next-token | L=1 | L=2 | Adaptive? |
|---|---|---|---|---|
| Standard ORN | **3.94** | 6.33 | 7.75 | No — needs all layers |
| PI-ORN (multi-horizon only) | 6.12 | 4.47 | **5.03** | Yes — but poor next-token |
| Hybrid (70/30) | 4.05 | **4.24** | 5.21 | **Yes — with good next-token** |

The key finding: the Standard ORN's single-layer prediction is 6.33 (near random)—it needs all 4 layers for anything useful. The Hybrid's single-layer prediction is 4.24—a competent prediction from depth 1 alone, with further depth available for hard tokens. The cost: only 0.11 nats worse on next-token (4.05 vs 3.94).

This enables a **practical form of adaptive compute**: run one layer, check confidence, stop if sufficient, continue if not. The multi-horizon objective teaches the model that each depth should produce a useful prediction—not just contribute to a final refinement.

**The role of perturbative corrections.** The 0.11 nat gap between Hybrid and Standard suggests a tension: the orbital path is shared between multi-horizon structure and next-token accuracy. The perturbative corrections are designed precisely for this: the orbital core handles the multi-horizon geometry (which tokens relate at different time scales), while the corrections sharpen the immediate prediction (which specific token comes next, given this context). A Hybrid ORN *with* perturbative corrections would use the orbital path for planning and the corrections for execution—separating the "what could happen" (multi-horizon) from the "what will happen" (next-token). This experiment is designed and ready to run (Exp PI-ORN+Corr).

The deeper question is whether the ORN's three-component decomposition—$\mathbf{M}$ for structure, response heads for perception, corrections for action—can support *simultaneous* planning and execution. Standard training makes all components serve one prediction (execution only). Multi-horizon training makes the orbital path serve planning (at a cost to execution). The corrections are the mechanism for recovering execution quality without destroying the planning capability, because they target the *immediate* output without altering the orbital trajectory that encodes multi-horizon structure.

If this works, the ORN is not just an efficient transformer variant—it is an architecture with a principled separation between world-modelling (the orbital path plans ahead through the semigroup) and token-level prediction (the corrections sharpen the plan into a specific next token). The response heads mediate between the two: they read the multi-horizon field state and extract what is needed for the current depth's prediction. This is the "adaptive compute" vision of this paper made concrete: not just "think longer on hard tokens" but "plan at every depth and execute at the final depth, with the gate controlling how much execution refinement to apply."

## 7 Connections

### 7.1 To the Stigmergic Residue Programme

This paper extends the shared-geometry programme from fixed to variable depth. The first paper establishes that $\mathbf{M}$ can replace per-layer coupling; this paper asks whether $\mathbf{M}$ can also replace the fixed computational budget. Together, they propose an architecture where *both*

the coupling rule and the computational depth are properties of the field geometry rather than architectural constants.

### 7.2 To Deep Equilibrium Models

DEQs [2] find fixed points of weight-tied layers using root-finding algorithms (Anderson acceleration, Broyden's method). Our approach is complementary: rather than solving for the fixed point implicitly, we iterate explicitly and halt when converged. The advantage is simplicity and interpretability (we can inspect the orbit); the disadvantage is that explicit iteration may be slower than implicit fixed-point solving for deep equilibria.

### 7.3 To JEPA and Latent World Models

LeCun's JEPA programme [8] proposes world models that predict in representation space rather than observation space. V-JEPA [9] applies this to video: a latent dynamics model predicts future frame representations without generating pixels.

The $T$-loop in ORN is structurally analogous: iterate a shared operator in latent space (the residual field) to refine representations—"imagination" without observation-level decoding. The key difference is that JEPA's predictor is a directional function (context $\rightarrow$ target), while $\mathbf{M}$ is a relational metric (all-to-all coupling). JEPA iterates forward in time; ORN iterates in depth. But both are running dynamics in a learned geometry without generating raw observations at each step.

V-JEPA's success on video prediction—learning useful representations through pure latent dynamics—is indirect evidence that our variable-$T$ programme should work. If latent iteration can capture temporal dynamics in video, it should capture computational depth dynamics in language. LLM-JEPA [10] recently confirmed that embedding-space prediction improves language models when used as a hybrid objective alongside autoregressive training, without degrading generation quality. This validates the principle that latent-space dynamics and token-level generation are complementary, not competing—the same relationship between our $T$-loop (latent iteration) and $R$-loop (token generation).

LeCun's full architecture includes a "configurator" for binding specific identities, paralleling our ontological pathway. The convergence crossover at step 2800 in our Experiment 3 (shared geometry starts slow, then overtakes) may reflect the same developmental asymmetry that JEPA exhibits: the geometric structure must crystallise before identity binding becomes useful.

### 7.4 To Biological Cognition

The variable-depth model maps onto the observation that biological thinking takes variable time. Fast System 1 responses (Kahneman) correspond to shallow orbits; slow System 2 responses correspond to deep orbits in the same geometry. The shared $\mathbf{M}$ is the cortical connectivity; the variable iteration is the recurrent processing time; the convergence criterion is the "feeling of knowing" that terminates deliberation.

We do not claim this is how brains work. We claim it is a useful computational model that is inspired by the same constraints: a fixed geometry, variable computation, and convergence as the natural halting condition.

## 8 How "Thinking" Works in Current Systems

Before developing the ORN's approach to deliberative computation, it is worth examining how modern language models implement "thinking." The answer is surprisingly simple: there is no architectural change.

### 8.1 Chain-of-Thought as Token Generation

Models such as OpenAI's o1/o3 and DeepSeek-R1 "think" by generating a long chain of reasoning tokens—actual text like "Let me consider... Wait, that contradicts... So the answer must be..."—before producing the final answer. Each thinking token is generated by the *same* $L$-layer forward pass as any other token. There is no recurrence, no extra iterations, no latent-space computation. The model is literally talking to itself in token space.

The innovation is in training, not architecture: reinforcement learning from outcome feedback teaches the model that generating intermediate reasoning tokens leads to better final answers. The model learns to produce useful thoughts by trial and error, not by any structural change to the computation.

### 8.2 What This Costs

- Each thinking token requires a full forward pass ($L$ layers, full attention over the growing context).
- Reasoning must be serialised into language—the model cannot think in parallel or in non-linguistic representations.
- The context window limits how long the model can think.
- The thinking is confined to the vocabulary's expressiveness: each step carries at most $\log_2 V$ bits of information (a token choice), whereas a latent representation carries $d$ continuous dimensions.

### 8.3 Latent Alternatives

COCONUT [11] and MARCOS [14] propose latent-space reasoning: instead of generating text tokens, feed the hidden state back through the network. Each "thought" is a $d$-dimensional vector, not a vocabulary-sized distribution. This is cheaper (no token sampling or embedding lookup), richer ($d$ dimensions vs $\log_2 V$ bits per step), and can represent multiple possibilities simultaneously (COCONUT's BFS-like behaviour). But it is not inspectable—you cannot read a hidden state.

Looped transformers (Ouro/LoopLM [12], Mixture-of-Recursions [16]) share parameters across iterations, giving the recurrence a principled structure: each pass through the shared layers is an iteration of the same operator. At 1.4B–2.6B parameters, Ouro matches models 2–3× larger, with the advantage concentrated on multi-hop reasoning tasks—exactly where "thinking longer" should help.

### 8.4 What the ORN Adds

In a standard transformer, there is no principled reason for chain-of-thought steps to be *self-consistent*. Each forward pass is independent; the network has no dynamics that compose lawfully. The reasoning quality depends entirely on the training distribution of reasoning traces.

The ORN's shared $\mathbf{M}$ defines actual dynamics: a semigroup with a coupling geometry. Extra iterations through $\mathbf{M}$ are iterations of a *dynamical system*, not just repeated function application. The Chapman-Kolmogorov property (when trained for it) guarantees that these iterations compose lawfully: the state reached in $s{+}t$ steps is consistent with reaching an intermediate state in $s$ steps and then continuing for $t$ more. This is a mathematical guarantee about the structure of the reasoning trajectory that token-space chain-of-thought cannot provide.

The spectral partition conjecture (Section 10) goes further: prediction and reasoning could live in different spectral regions of the same $\mathbf{M}$. Fast prediction uses the top eigenvectors (high energy, convergent). Slow reasoning uses the lower eigenvectors (low energy, composable, divergent). One network, one $\mathbf{M}$, two capabilities addressed through different spectral channels—with no mode switching, no special tokens, and no architectural changes.

# 9 Two Computational Modes: Prediction and Integration

The experiments in this paper and the companion path integral paper reveal a tension between two fundamentally different ways a shared-geometry network can compute.

## 9.1 Mode 1: Prediction (Convergent)

Standard next-token training optimises each layer to refine a single trajectory through representation space toward the most likely continuation. The layers homogenise: each applies the same coupling rule $\mathbf{M}$ to a progressively smoother field, and the response heads at each depth sharpen the prediction. This is convergent computation—the orbit contracts toward a fixed point (the predicted token).

This is what the ORN does well. Val loss 3.35 at 91M params. Fast, automatic, System 1.

## 9.2 Mode 2: Integration (Divergent)

The semigroup and path integral analysis showed a different kind of computation. Chapman-Kolmogorov composability asks: does $P(A \to C) = \sum_B P(A \to B) \cdot P(B \to C)$? This requires maintaining awareness of *all* intermediate states $B$, not collapsing to the most likely one. Our experiments showed that CK loss achieves composability (0.99 correlation) but the network doesn't *use* composition for language—because prediction training rewards convergence, not exploration.

Multi-horizon training (predicting 1, 2, 4, 8 steps ahead simultaneously) pushes toward integration: the model must represent not just "what comes next" but "what the space of futures looks like." Our results showed this doesn't help with perturbative corrections at current scale—the planning/execution tension cannot be resolved by additive corrections to a convergent backbone.

The path integral network (G6) *can* compute exact physics (Ising thermodynamics to 3–4 significant figures) when trained with an integration objective. But it requires a different architecture from the one optimised for prediction.

## 9.3 The Analogy to Thinking

| | **Prediction (System 1)** | **Integration (System 2)** |
|---|---|---|
| Question | What comes next? | What could happen? |
| Computation | Contract to mode | Maintain distribution over paths |
| Depth | Fixed (converge in $L$ layers) | Variable (iterate until explored enough) |
| Coupling role | $\mathbf{M}$ routes attention to relevant tokens | $\mathbf{M}$ defines the space of possible transitions |
| Output | Point estimate (next token) | Weighted aggregate over futures |
| Analogy | Perception, fluent speech | Planning, reasoning, creativity |

Prediction asks "what's most likely?" Integration asks "what does the full landscape look like?" Both use the same coupling geometry $\mathbf{M}$—the rules of the game don't change. What changes is how the network traverses $\mathbf{M}$'s geometry: convergent (collapse to attractor) vs divergent (explore the basin structure).

## 9.4 Where Integration Would Help in AI

**Planning and search.** Current LLMs plan by predicting one token at a time. They cannot natively evaluate "if I say $X$, they respond $Y$, then I need $Z$"—that requires composing transitions, which is exactly what the CK property formalises. An integration mode would maintain the tree of possibilities and weight them, replacing the external hack of tree-of-thought prompting with a native architectural capability.

**Reasoning under uncertainty.** Prediction gives the mode (most likely outcome). Integration gives the expectation (average over outcomes weighted by probability). For decisions under uncertainty, the expectation is what matters: "what is the expected value of this action across all possible consequences?"

**Counterfactual reasoning.** "What would have happened if…?" requires considering paths not taken. The prediction network knows only the path it is on. An integrator maintains awareness of paths not taken and can evaluate them.

**Creative and divergent generation.** A prediction network converges to the most likely continuation—useful for fluent text, not for novel ideas. An integration network maintains multiple continuations and can select based on criteria other than likelihood: novelty, coherence with a distant goal, emotional arc, or structural balance (like a musical resolution arriving at the right moment).

### 9.5 The Compose-Then-Predict Conjecture

The three-architecture proposals above (dual-head, U-Net, mode-switched) are variations on a single idea. We now refine it into a precise conjecture grounded in the semigroup analysis and supported by recent work on latent reasoning.

**Conjecture 2** (Compose-then-predict)**.** *An ORN can operate in two sequential phases using the* same $\mathbf{M}$ *and response heads:*

1. ***Compose phase*** *($K$ iterations): iterate* $\mathbf{M}$ *with a Chapman-Kolmogorov consistency loss. The residual field* expands *to represent the space of reachable states. The hidden state after $K$ compose iterations encodes not "the next token" but "the landscape of possible futures."*
2. ***Predict phase*** *($J$ iterations): iterate* $\mathbf{M}$ *with standard next-token loss. The residual field* contracts *from the landscape to a point estimate. The rich path structure built during compose informs a better prediction than predict-only would achieve.*

*No new parameters. No architectural changes. No halting mechanism. The same ORN is composed with itself: first as an integrator, then as a predictor.*

**The mathematical structure.** Let $h_0$ be the input embedding and $f(\mathbf{M}, h)$ the ORN layer function (shared coupling + per-layer response heads + corrections).

$$\text{Compose:} \quad h_{k+1} = h_k + f(\mathbf{M}, h_k), \quad k = 0, \ldots, K-1, \quad \mathcal{L}_{\text{CK}} \text{ on } h_K \tag{1}$$

$$\text{Predict:} \quad h_{K+j+1} = h_{K+j} + f(\mathbf{M}, h_{K+j}), \quad j = 0, \ldots, J-1, \quad \mathcal{L}_{\text{CE}} \text{ on } h_{K+J} \tag{2}$$

The compose loss $\mathcal{L}_{\text{CK}}$ enforces the Chapman-Kolmogorov property: the 2-step transition from $h_0$ to $h_2$ must equal the composition of 1-step transitions $h_0 \to h_1 \to h_2$. This forces $h_K$ to encode the full transition structure, not just the endpoint. The predict loss $\mathcal{L}_{\text{CE}}$ is standard cross-entropy on the output. Total loss: $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \mathcal{L}_{\text{CK}}$.

**Critical: selective gradient routing.** A naive implementation applies both losses to all parameters. Preliminary experiments (Exp I-1 dry run) showed this is catastrophic: CK loss warps the response heads away from prediction, producing good composability but destroying standard-mode performance (val loss 0.51 in native mode, 2.40 in standard mode).

The fix: **CK gradients update only $A$ and $B$ (the coupling geometry). CE gradients update everything.** Composability is a property of the coupling geometry: "transitions compose correctly" is a statement about $\mathbf{M}$, not about what the response heads do with the coupled information. Response heads must be free to specialise for prediction without interference from the CK objective. $\mathbf{M}$ receives both CE and CK gradients—the coupling geometry

must support both phases. Response heads receive only CE gradients—they remain optimised for prediction.

This is a separation of concerns that mirrors the ORN's architecture: **M** defines the relational rules (shared across phases), response heads define the content response (prediction-specific).

**Connection to consistency models and diffusion.** Our CK loss is the discrete-layer analogue of the consistency condition in Song et al.'s consistency models [18]: both enforce that the model's output is self-consistent across different numbers of transition steps. Consistency models train a diffusion process so that $f(x_t) = f(x_{t'})$ for any two points on the same trajectory; our CK loss trains **M**'s semigroup so that $T \circ T$ matches $T^2$.

The connection to diffusion is deeper than technical. Diffusion models start with noise (a fuzzy, unstructured image) and progressively refine detail through iterative denoising—each step sharpens the picture while maintaining consistency with the global structure. The compose phase of compose-then-predict is analogous: start with a fuzzy representation of possible futures (the initial residual field), iterate through **M** to build consistent path structure, then the predict phase reads the clarified landscape and makes a decision. Slow thinking IS denoising: starting with a vague sense of the answer and progressively refining it through deliberation, where each refinement step must be consistent with the relational structure (**M**) of the problem.

**Why this resolves the composability contradiction.** Our semigroup experiments (Section **??**) showed that CK loss achieves 0.99 composability but hurts prediction when applied to all parameters. Selective gradient routing resolves this: the compose phase shapes **M**'s geometry for path consistency (via CK loss on $A, B$), while the predict phase uses that geometry for convergent prediction (via CE loss on everything). The response heads never see CK gradients, so they remain prediction-optimised. **M** sees both, so it learns a geometry that supports both composability and prediction—the universal coupling rule that is the ORN's central claim.

**Connection to COCONUT and latent reasoning.** Meta's Chain of Continuous Thought (COCONUT) [11] feeds the last hidden state back as the next input embedding, iterating in continuous latent space rather than decoding to tokens. Key finding: continuous thoughts can encode *multiple alternative next steps simultaneously*, enabling breadth-first exploration. COCONUT achieves 97% on ProsQA vs 77.5% for chain-of-thought.

The compose phase of our architecture is structurally identical to COCONUT's latent iteration: the hidden state is re-fed through the same network. The difference is the objective: COCONUT uses prediction loss throughout; we propose CK loss during compose and prediction loss during predict. This separation may enable the BFS-like exploration COCONUT discovers but with a principled mathematical foundation (the semigroup's Chapman-Kolmogorov property) rather than emergent behavior.

**Connection to Ouro/LoopLM.** ByteDance's Ouro [12] demonstrates that shared-parameter looped transformers achieve 2–3× parameter efficiency at 1.4B–2.6B parameters, with the advantage coming from superior *knowledge manipulation* (multi-hop reasoning) rather than knowledge storage. This validates the core ORN hypothesis at scale: shared parameters + variable iteration beats fixed-depth models on tasks requiring composition. The compose-then-predict architecture is a principled way to control what the extra iterations do: compose builds the reasoning scaffold, predict reads it.

**Connection to multi-token prediction.** Meta's multi-token prediction [13] trains shared-trunk transformers to predict 1, 2, 4, 8 tokens ahead simultaneously, finding that multi-horizon training improves single-token inference. Our dual-head experiment confirms this: adding a multi-horizon head to the ORN improved prediction by 1.2% without changing the backbone.

The compose phase generalises this: instead of separate prediction heads per horizon, the compose iterations build a single representation that encodes all horizons, which the predict phase reads.

**The neuroscience parallel.** The two-phase structure maps onto two well-documented modes of cortical processing:

- **Recurrent processing** (Lamme & Roelfsema, 2000 [3]): initial feedforward sweep (fast, convergent) followed by recurrent loops (slow, integrative) through the same cortical circuits. The feedforward sweep gives a rough categorisation; the recurrent loops resolve ambiguity, bind features, and support conscious awareness.
- **Default mode network**: the brain's "compose" mode. Active during rest, mind-wandering, planning, and counterfactual reasoning. Suppressed during focused task execution ("predict" mode). Same cortical geometry, different dynamics.
- **Hippocampal replay**: offline reactivation of experienced trajectories, believed to support planning by mentally simulating future paths. The compose phase is a forward-pass analogue: building a representation of possible trajectories before committing to one.

In all three biological cases, the same neural substrate supports both fast convergent processing and slow integrative processing. The mode switch is not architectural—it is dynamical. This is exactly the compose-then-predict conjecture: same $\mathbf{M}$, different phases.

### 9.6 Key Insight: Composition, Not Adaptation

The compose-then-predict architecture adds no parameters and changes no weights. It is pure *composition*—the ORN composed with itself. This is a design principle: when you have a good component, duplicate and reuse it rather than modifying it. Evolution works this way (gene duplication, not redesign). The ribosome doesn't have a "think harder" mode; the cell runs more ribosomes on the same mRNA.

Alternative approaches—adaptive halting (Architecture B), mode tokens, separate integration heads—all require new parameters or training objectives that change the base ORN. Compose-then-predict preserves the ORN exactly as trained and asks: does the computation it already performs, when iterated, produce richer representations?

If yes, the ORN's orbit through $\mathbf{M}$'s geometry has a natural multi-scale structure: short orbits for fast prediction, long orbits for deep reasoning, with no architectural distinction between the two. The "horizon" in "orbits without horizon" is not a design parameter—it is an emergent property of the input difficulty and the time available for computation.

### 9.7 Proposed Experiments

**Exp I-0: Zero-shot compose-then-predict.** Take a trained ORN ($L$=6) and at inference, run it twice: first pass = compose (no output), second pass = predict (output from final layer). Input to the second pass = hidden state output of the first pass. Compare val loss: 1 pass (standard) vs 2 passes (compose+predict) vs 2 passes (both predict, i.e. just $2L$ depth with prediction loss). If compose+predict beats double-predict at the same total compute, the two-phase structure adds something beyond depth. **This requires no retraining.**

**Exp I-1: Trained compose-then-predict.** Train an ORN with $K$+$J$ total iterations per forward pass. Apply CK loss on iterations $1, \ldots, K$ (compose) and prediction loss on iteration $K$+$J$ (predict). Vary $K$ and $J$: ($K$=0, $J$=6) is the standard ORN; ($K$=3, $J$=3) splits evenly; ($K$=6, $J$=6) doubles total depth with half compose, half predict. Measure: val loss, multi-horizon accuracy, gate magnitudes during compose vs predict phases.

**Exp I-2: Compose depth scaling.** Fix $J$=6 (predict) and vary $K = 0, 1, 2, 4, 8$ (compose). Plot val loss vs $K$. If loss improves then plateaus, the plateau defines the natural "thinking depth" for the data. If loss degrades, compose iterations harm the residual field (the orbit diverges). The correction gates during compose are the diagnostic: if they activate strongly, the network is using corrections to maintain path diversity; if they are silent, the compose iterations are inert.

**Exp I-3: Does compose help hard tokens?** Partition tokens by prediction difficulty (high cross-entropy under the baseline model). Measure the improvement from compose iterations on easy vs hard tokens. If compose selectively helps hard tokens, the architecture naturally implements adaptive compute without an explicit halting mechanism—you simply give more compose iterations to hard inputs.

# 10 Orbit-Conditioned Prediction

The compose-then-predict architecture treats the two phases as sequential: compose enriches the representation, predict reads it. But the compose phase produces more than a single enriched state—it produces an **orbit**: a trajectory of states $\{T(k)h_0 : k = 1, \ldots, K\}$ through the representation space defined by $\mathbf{M}$'s semigroup. This orbit encodes the *space of reachable states from* $h_0$, not a single prediction.

**Conjecture 3** (Orbit-conditioned prediction)**.** *A prediction network can use semigroup orbit points as context, alongside or instead of token embeddings. The orbit points serve as a "structural prompt"—they tell the prediction network what relational patterns are active, what transitions are available, and what states are reachable, without committing to a specific sequence of tokens.*

*Formally: let* $\mathcal{O}(h_0, K) = \{T(1)h_0, T(2)h_0, \ldots, T(K)h_0\}$ *be the* $K$*-step orbit of* $h_0$ *under the semigroup generated by* $\mathbf{M}$*. The prediction network receives as context:*

$$context = [\underbrace{e_1, e_2, \ldots, e_n}_{\text{token prompt (optional)}}, \underbrace{T(1)h_0, T(2)h_0, \ldots, T(K)h_0}_{\text{orbit context}}]$$

*and predicts the next token conditioned on both the explicit history (token prompt) and the implicit relational landscape (orbit).*

**Why orbit points work as context.** The orbit points are $d$-dimensional vectors in the same residual stream space as token embeddings. The prediction network already knows how to attend to things in this space—it was trained on token embeddings that live there. An orbit point $T(k)h_0$ is not a token; it is a representation of "what the coupling geometry looks like $k$ steps from here." The prediction network attends to orbit points just as it attends to tokens, but extracts relational structure rather than lexical content.

If the orbit is CK-consistent ($T(s{+}t) = T(s) \circ T(t)$), the orbit points have coherent structure: the jump from $T(2)h$ to $T(5)h$ is the same transition as $T(3)$ applied to $T(2)h$. This consistency means the prediction network sees a lawful trajectory, not random points—it can interpolate and extrapolate within the orbit.

**Prompt vs orbit: two kinds of context.** A token prompt is a specific path through language: "The cat sat on the..." It constrains prediction to continuations of that specific history.

An orbit is a *family* of paths through representation space. Different orbit points $T(k)h_0$ explore different regions of the reachable state space from the same seed $h_0$. Feeding orbit points as context gives the prediction network awareness of the path family, not just one path. This

is the divergent quality: the orbit says "here are the states this system can reach" rather than "here is the one thing that happened."

A prediction conditioned on both prompt AND orbit would combine specific history (what happened) with structural awareness (what could happen). The prompt anchors the prediction; the orbit broadens it.

**The encoding problem.** The orbit must be produced by a network trained for composability (CK loss), while the prediction network is trained for convergence (CE loss). Our experiments showed these objectives conflict when applied to the same parameters. The orbit-conditioned architecture separates them cleanly:

- A **compose network** (own response heads, possibly own $\mathbf{M}_C$) trained with CK loss produces orbit points.
- A **predict network** (own response heads, own $\mathbf{M}_P$, or shared $\mathbf{M}$) trained with CE loss reads orbit points as context and predicts.
- The predict network also works on raw prompts without orbit context (fast mode).
- The interface between them is the residual stream space—orbit points and token embeddings are the same dimensionality.

Whether $\mathbf{M}_C = \mathbf{M}_P$ (shared geometry) or $\mathbf{M}_C \neq \mathbf{M}_P$ (separate geometries) is an empirical question. Our results suggest that CK and CE require different geometries when applied to the same M. But if compose produces orbit points that are useful as context for a CE-trained predictor, the two geometries need not be identical—they just need to produce representations in the same space.

**Connection to diffusion and denoising.** The orbit is a discretised trajectory through $\mathbf{M}$'s vector field. In the denoising analogy, the orbit starts from a noisy (high-entropy) state and each step refines it. The predict network reads the refined state and produces a sharp output. But unlike diffusion, the orbit is produced by a CK-consistent semigroup, so the trajectory has lawful structure that the predict network can exploit—it is not random noise being progressively removed, but a structured exploration of the reachable state space being progressively resolved.

**What would confirm this.**

1. CK diagnostics (Exp I-4, queued) show that CK-trained orbits preserve information and maintain non-trivial structure (not collapsed, not chaotic).
2. A predict network trained on orbit context achieves lower val loss than one trained on token context alone.
3. More orbit points (longer orbits) improve prediction, then plateau—the plateau defines the natural "thinking depth."
4. The orbit-conditioned predictor handles ambiguous or difficult inputs better than the baseline, because the orbit provides structural context that a token prompt does not.

### 10.1 Decision Tree: What CK Diagnostics Could Show

The CK diagnostics experiment (Exp I-4) measures what CK-trained representations actually compute. The outcome determines whether orbit-conditioned prediction is viable and what modifications are needed.

**1. Collapse** (variance $\to$ 0, all inputs map to same state). CK is trivially satisfied by forgetting. The semigroup contracts to a fixed point—composable but useless. **Action:** CK loss needs an information-preservation regulariser (contrastive loss or entropy maximisation) to prevent trivial solutions. Alternatively, abandon CK and seek a different composability objective.

**2. Information preserved, orbits converge** (inputs distinguishable but similarity increases with iteration). CK has learned a well-behaved contractive semigroup—composable transitions that converge to an attractor. This is "composable prediction"—not the divergent thinking we wanted, but still potentially useful. **Action:** Orbit points encode a smooth convergence trajectory. The prediction network could exploit "how far along the convergence path" each point is. Useful for the orbit-conditioned architecture but does not provide divergent structural context.

**3. Information preserved, orbits parallel** (different inputs stay different, similarity constant across iterations). The best case. CK has learned volume-preserving (isometric) dynamics—the semigroup maps different starting points to different regions without convergence or divergence. **Action:** These orbit points ARE the structural landscape we want. Each point represents a different reachable state from the same seed. Feed them as context to the prediction network. This directly enables the orbit-conditioned prediction conjecture.

**4. Orbits diverge** (similarity decreases, entropy increases with iteration). CK has learned expansive dynamics—the semigroup amplifies differences. Orbit points become noisy and hard for the prediction network to use. **Action:** Add a mild contraction penalty alongside CK to keep orbits bounded. The goal: composable dynamics that explore without exploding.

**5. Approximately linear** ($T(h_1 + h_2) \approx T(h_1) + T(h_2)$). CK is trivially satisfied by a linear map. The semigroup is matrix exponentiation—composable but geometrically uninteresting. The nonlinearity of the response heads is being suppressed. **Action:** Increase CK training or add a nonlinearity encouragement loss. Linear orbits are just rotated/scaled inputs—they provide no structural enrichment.

**6. Structured basins** (entropy stable, orbits form clusters or attractors corresponding to data categories). The most interesting outcome. CK dynamics create a landscape of basins—regions of representation space that correspond to meaningful structure in the data. **Action:** The orbit structure IS the relational landscape. Orbit points from different basins provide the prediction network with awareness of alternatives. This most directly supports the "thinking as exploring the landscape" interpretation.

The critical diagnostic pair is **information preservation + orbit structure**. If information is preserved and orbits neither converge nor diverge, CK produces useful structural context. If either fails (collapse or convergence), CK needs modification before the orbit-conditioned architecture is viable.

### 10.2 Diagnostic Results

The diagnostics returned clear answers (see semigroup companion paper for full tables):

- **CK-only:** Perfect composability (error ≈0) but information destroyed (dist_ratio 0.04). All inputs converge to the same max-entropy trajectory. *Composable but useless.*
- **CK(M)+Predict:** Information preserved (0.73), orbits diverge (cosine 0.81→0.50), entropy increases (0.49→0.71). *Divergent, information-preserving — the qualitative behaviour we want.*

**Consistency does not create meaning — it preserves it.** CK-only found the trivial solution because nothing anchored the representations to meaningful structure. This parallels consistency model distillation [18]: you do not train a diffusion process from scratch with consistency loss; you distill a pre-trained model into a consistent version.

### 10.3 Curriculum Training and the Fundamental Tension

We tested a curriculum: CE-only pre-training, then CE+CK with ramping $\lambda$. Five configurations at $d$=128, 8000 steps:

| Config | Val | vs base | CK err | Ent Δ | Orbit Δ |
|---|---|---|---|---|---|
| A: Predict-only | 0.478 | — | 0.814 | −0.07 | −0.14 |
| B: Curriculum $\lambda \to 0.1$ | 0.491 | +2.6% | **0.693** | **+0.15** | −0.11 |
| E: Curriculum $\lambda \to 0.01$ | 0.481 | +0.5% | 0.814 | −0.03 | −0.08 |
| D: CK from start | 0.483 | +0.9% | 0.835 | +0.11 | −0.18 |

None achieved lower CK AND preserved prediction. Config B reduced CK error by 15% with entropy increasing (+0.15), but prediction degraded 2.6%.

**Reframing: the tension IS the value.** Treating the 2.6% prediction cost as a failure misses the point. You cannot think fast and slow simultaneously—this is given. The prediction cost IS the slow thinking: the network maintains awareness of possibilities (entropy +0.15) rather than collapsing to the most likely token (entropy −0.07 for predict-only). Different inputs stay more distinguishable (orbits −0.11 vs −0.14). The representations are richer, more divergent, and structurally more lawful (CK error −15%).

Obsessing over prediction loss is precisely what prevents the field from building world models that can reason. A model that loses 2.6% on next-token prediction but gains 15% in transition consistency and maintains higher representational entropy is not worse—it is *differently capable*.

**The scaling argument.** At toy scale ($d$=128, 8000 steps), the 2.6% cost is measurable. At V2 scale ($d$=512, millions of steps), three factors reduce it:

1. **M** has more spectral capacity—composability and prediction can occupy different eigenvectors without competing.
2. More training steps mean the curriculum's Phase 2 is a smaller fraction of total training.
3. Fine-tuning with CE-only after curriculum training can recover the prediction loss while keeping the composable geometry. Response heads adapt to the richer **M**; the geometry stays.

The prediction cost is a training-time investment, not a permanent architectural limitation. The return on that investment is a world model whose dynamics are self-consistent, whose representations maintain structural diversity, and whose coupling geometry supports both convergent prediction and divergent reasoning.

## 11 Imagination, Not Planning

The compose-then-predict architecture and the complex depth hypothesis point toward a capability that is not planning in the classical AI sense, but something more fundamental: **imagination**.

### 11.1 The Distinction

Planning asks: "what sequence of actions leads to a goal?" It requires search, constraint satisfaction, and backward reasoning from the goal to the present. Current language models approximate planning by generating chain-of-thought reasoning in token space, but this is search by serial enumeration, not structural foresight.

Imagination asks: "what could happen from here?" It does not require a goal. It requires a world model that can generate *plausible futures*, weighted by their likelihood, without committing to any single trajectory. Imagination is the prerequisite for planning (you must imagine

possibilities before you can choose among them), but it is valuable independently: prediction, story generation, empathy, and scientific reasoning all require imagining what is not yet present.

### 11.2 The Imaginary Spectrum

The connection to the complex depth hypothesis is not merely linguistic. $\mathbf{M}$'s eigenvalues $\lambda_k$ have real and imaginary parts. The imaginary parts produce oscillatory dynamics: $e^{i\beta\lambda_k}$ rotates through the eigenspace without converging. This is literally "imagining" in the mathematical sense: traversing the space of possibilities without collapsing to one.

- **Real depth** ($t = \tau$): causal evolution, convergent prediction, "what will happen."
- **Imaginary depth** ($t = i\beta$): oscillatory exploration, divergent imagination, "what could happen."
- The **imaginary** Koopman modes are the **imagination** modes.

The 89% of V1's eigenvalues that have imaginary parts are not noise. They are the oscillatory dynamics that enable the model to represent multiple possible futures simultaneously. Prediction training uses their real parts (convergence). Imagination would use their imaginary parts (exploration).

### 11.3 Testing Imagination: Distribution Quality, Not Token Accuracy

The right test for imagination is not "did the model generate the correct next word" (that is prediction accuracy). It is: **does the model's probability distribution over futures correctly reflect narrative plausibility?**

**The likelihood ratio test.** Given a story at sentence $k$, the model assigns probability $P(\text{continuation} \mid \text{context})$ to every possible continuation. A good world model assigns:

- High probability to the actual continuation and to narratively plausible alternatives.
- Low probability to narratively implausible continuations.

The metric: $\text{LR}(k) = P(\text{actual continuation at horizon } k) \;/\; P(\text{random continuation})$. This measures the *quality of the distribution*, not the quality of the argmax. A model that produces fluent but poorly calibrated text (high token accuracy, flat distribution over futures) has poor imagination. A model with correctly weighted futures (sharp distribution concentrated on plausible outcomes) has rich imagination.

**Multi-horizon consistency.** Measure LR at horizons $k = 1, 4, 8$ sentences ahead. A model with composable dynamics (CORN) should maintain high LR at long horizons because $T(k) = T(1)^k$: the $k$-step prediction is structurally consistent with the 1-step prediction composed $k$ times.

An ORN (prediction-only) may have high LR at $k$=1 but declining LR at $k$=4, 8 because it has no mechanism for ensuring multi-step consistency.

If CORN maintains higher LR at long horizons, it is "imagining further ahead" with better calibration. This is not planning (there is no goal). It is *structured imagination*: a world model that correctly weights distant futures because its dynamics compose.

**Sampling diversity as imagination richness.** Sample 100 continuations from the model by sampling (not argmax). Score: how many distinct narratively-consistent continuations does the model produce? A model that produces 100 copies of the same continuation has narrow imagination (collapsed to the mode). A model that produces 100 *different but all plausible* continuations has rich imagination (broad, well-calibrated distribution).

CORN's divergent dynamics (higher entropy, less convergent orbits) predict richer imagination: more distinct plausible continuations, with the semigroup property ensuring they are all internally consistent.

### 11.4 Proposed Experiment: Imagination Quality on TinyStories

1. Train ORN and CORN on TinyStories (same architecture, same budget).
2. Take 1000 test stories. At each sentence boundary, extract the model's hidden state.
3. Compute the model's log-probability for:
   - The actual next sentence (true future).
   - 10 random next-sentences from other stories (false futures).
4. The **imagination score** at horizon $k$: the average log-likelihood ratio $\log P(\text{true}) - \log P(\text{false})$ at $k$ sentences ahead.
5. Compare ORN vs CORN at $k = 1, 4, 8$.
6. Also: sample 50 continuations from each model. Count the number of distinct narratively-plausible continuations (human evaluation or automated coherence scoring).

**What would confirm.** CORN has higher imagination score at $k \geq 4$ (better calibrated long-range futures) while matching ORN at $k$=1 (same short-range prediction). CORN produces more diverse plausible continuations (richer imagination). These results together would show that composable semigroup dynamics improve not just composability metrics (SG error, rank) but the *functional quality of the world model* as measured by the structure of its probability landscape.

**What this is not.** This is not planning. It does not demonstrate goal-directed behaviour, constraint satisfaction, or search. It demonstrates that the model's internal representation of "what could happen" is well-structured and correctly weighted, which is the foundation on which planning could be built. Imagination is the canvas; planning is the drawing. We aim to show the canvas is well-prepared.

## References


[1] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. Universal Transformers. *ICLR*, 2019.

[2] S. Bai, J. Z. Kolter, and V. Koltun. Deep Equilibrium Models. *NeurIPS*, 2019.

[3] V. A. F. Lamme and P. R. Roelfsema. The distinct modes of vision offered by feedforward and recurrent processing. *Trends in Neurosciences*, 23(11):571–579, 2000.

[4] J. Pérez, J. Marinković, and P. Barceló. On the Turing Completeness of Modern Neural Network Architectures. *ICLR*, 2019.

[5] J. Pérez, P. Barceló, and J. Marinković. Attention is Turing Complete. *Journal of Machine Learning Research*, 22(75):1–35, 2021.

[6] S. Bhatt, E. Hovy, and M. Lipson. Turing Completeness of Bounded-Precision Recurrent Neural Networks. *NeurIPS*, 2021.

[7] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural Ordinary Differential Equations. *NeurIPS*, 2018.

[8] Y. LeCun. A Path Towards Autonomous Machine Intelligence. *openreview.net*, 2022.

[9] A. Bardes et al. V-JEPA: Latent Video Prediction for Visual Representation Learning. *arXiv:2404.08471*, 2024.

[10] LLM-JEPA: Large Language Models Meet Joint Embedding Predictive Architectures. *arXiv:2509.14252*, 2025.

[11] H. Hao et al. Training Large Language Models to Reason in a Continuous Latent Space. *arXiv:2412.06769*, 2024.

[12] T. Fan et al. Ouro: Looped Language Models with Entropy-Regularized Adaptive Halting. *arXiv:2510.25741*, 2025.

[13] F. Gloeckle et al. Better & Faster Large Language Models via Multi-Token Prediction. *arXiv:2404.19737*, 2024.

[14] MARCOS: Modelling Reasoning as a Hidden Markov Chain of Continuous Thoughts. *arXiv:2509.25020*, 2025.

[15] S. Goyal et al. Think Before You Speak: Training Language Models with Pause Tokens. *ICLR*, 2024.

[16] Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *NeurIPS*, 2025.

[17] D. Raposo et al. Mixture-of-Depths: Dynamically Allocating Compute in Transformer-Based Language Models. *arXiv:2404.02258*, 2024.

[18] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. Consistency Models. *ICML*, 2023.

[19] G. Deletang, A. Ruoss, J. Grau-Moya, T. Genewein, L. K. Wenliang, E. Catt, C. Cundy, M. Hutter, S. Legg, J. Veness, and P. A. Ortega. Neural Networks and the Chomsky Hierarchy. *ICLR*, 2023.

[20] B. Lake and M. Baroni. Generalization without Systematicity: On the Compositional Skills of Sequence-to-Sequence Recurrent Networks. *ICML*, 2018.

[21] N. Kim and T. Linzen. COGS: A Compositional Generalization Challenge Based on Semantic Interpretation. *EMNLP*, 2020.

[22] B. Lake and M. Baroni. Human-like Systematic Generalization through a Meta-Learning Neural Network. *Nature*, 2023.

[23] S. Murty, P. Sharma, J. Andreas, and C. D. Manning. Pushdown Layers: Encoding Recursive Structure in Transformer Language Models. *EMNLP*, 2023.

[24] Stack Attention: Improving the Ability of Transformers to Model Hierarchical Patterns. *ICLR*, 2024.

[25] Emergent Stack Representations in Modeling Counter Languages Using Transformers. *arXiv:2502.01432*, 2025.

[26] J. Hewitt and C. D. Manning. A Structural Probe for Finding Syntax in Word Representations. *NAACL*, 2019.

[27] K. Krohn and J. Rhodes. Algebraic Theory of Machines. I. Prime Decomposition Theorem for Finite Semigroups and Machines. *Transactions of the AMS*, 1965.

[28] J.-E. Pin. Varieties of Formal Languages. Plenum, 1986.

[29] P. K. Rhee. Moving Beyond Next-Token Prediction: Transformers are Context-Sensitive Language Generators. *arXiv:2504.10845*, 2025.

[30] A Circuit for Predicting Hierarchical Structure In-Context in Large Language Models. *arXiv:2509.21534*, 2025.

[31] J. Schmidhuber. Learning to Control Fast-Weight Memories: An Alternative to Dynamic Recurrent Networks. *Neural Computation*, 4(1):131–139, 1992.

[32] I. Schlag, K. Irie, and J. Schmidhuber. Linear Transformers Are Secretly Fast Weight Programmers. *ICML*, 2021.

[33] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei. Learning to (Learn at Test Time): RNNs with Expressive Hidden States. *arXiv:2407.04620*, 2024.

[34] A. Behrouz and P. Zhong. Titans: Learning to Memorize at Test Time. *arXiv:2501.00663*, 2025.

[35] J. Liu, S. Elflein, O. Litany, Z. Gojcic, and R. Li. Test-Time Training with KV Binding Is Secretly Linear Attention. *arXiv:2602.21204*, 2026.

[36] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. *NeurIPS*, 2024.

[37] S. Yang et al. Gated Delta Networks: Improving Mamba2 with Delta Rule. *ICLR*, 2025.

[38] L. M. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, and S. Whiteson. Fast Context Adaptation via Meta-Learning. *ICML*, 2019.

[39] A. Behrouz and P. Zhong. Titans + MIRAS: Helping AI Have Long-Term Memory. *Google Research Blog*, December 2025.

[40] In-Place Test-Time Training. *ICLR*, 2026.

[41] LaCT: TTT Done Right—Large Chunk Test-Time Training. *arXiv:2505.23884*, 2025.

# 12 What Is CORN? What We Know, What We Don't

CORN (Composable Orbital Response Network) is **not an architecture**—it is a post-training recipe that adds a semigroup consistency loss to an existing ORN checkpoint. The goal: make $\mathbf{M}$'s coupling *composable*, so that iterating the dynamics for $s+t$ steps is equivalent to iterating for $s$ steps then $t$ steps: $T(s+t) \approx T(s) \circ T(t)$.

Standard ORN layers are a pipeline, not a semigroup. The V1 composition probe measured 3% top-1 agreement between $2 \times 12$ and $1 \times 24$ blocks. CORN training closes this gap by adding:

$$\mathcal{L}_{\text{SG}} = \|T(s+t,h) - T(s,T(t,h))\|^2$$

alongside the standard CE loss, with $t$-scaling through the coupling: $q = h \cdot (t \cdot A)$, $k = h \cdot (t \cdot B)$.

## 12.1 What CORN Has Delivered

**Multimodal: CORN wins 3 of 4 modalities.** At $d$=256, $L$=8, 20K steps on images + music + environmental audio + text, CORN beats the transformer on music ($-13.8\%$), environmental sounds ($-15.3\%$), and text ($-0.9\%$), trailing only on images ($+13.4\%$). The composable coupling dynamics handle cross-modal structure better than per-layer coupling.

**Free at scale.** At $d$=128, CORN costs 2.9% prediction quality. At $d$=256, the cost disappears: CORN matches ORN with zero prediction degradation and rank halving ($20 \rightarrow 10$). The prediction/composability tension is a small-scale artefact.

**Variable-depth convergence.** Standard fixed-depth ORN diverges beyond training depth (loss 3.36 at $T$=6, 4.10 at $T$=24). CORN-trained models stay stable: loss 3.37–3.39 across all depths, 10× better KL convergence. Extra iterations don't help (yet), but they don't hurt.

**Composition on the coupling space is architectural.** Realisation #101: shared **M** gives semigroup composition on the coupling space for free—99.6% fidelity without any semigroup loss. The CORN loss helps composition on the *hidden state* space, which is harder because the response heads introduce nonlinearity.

### 12.2 What CORN Has NOT Delivered

**Thinking.** No evidence that extra CORN iterations produce better predictions on hard tokens, multi-step reasoning, or planning. Variable-depth training prevents divergence but doesn't demonstrate that depth = thinking. This is the central open question.

**Multi-step denoising.** CORN is worse than ORN at denoising at small scale ($d$=96). Multi-step composition degrades 2× faster. The composability that helps multimodal prediction does not automatically help diffusion.

**Generation from noise.** CORN-generated images converge to the mean field (grey checkerboard). Denoising ≠ generation. The semigroup dynamics can refine existing structure but cannot create structure from nothing at current scale.

### 12.3 The Imagination Hypothesis

If CORN's composition property does more than compress the spectrum, what is it for? We propose: **imagination**—the ability to evaluate counterfactual trajectories by running the coupling dynamics beyond the standard depth.

**The mechanism.** In a standard ORN at $T$=$L$ layers, each token's representation is the endpoint of its orbit through **M**'s coupling geometry. If the dynamics compose, running to $T$=$2L$ is meaningful: the orbit continues through the same geometry, exploring further in coupling space. The orbit at $T$>$L$ represents "what would happen if I kept thinking"—a counterfactual trajectory that standard fixed-depth models cannot access.

**M**'s eigenspectrum determines what these extended orbits explore:

- **Real eigenvalues (dominant):** contracting modes that converge to fixed points. Extended iteration refines the prediction—same answer, higher confidence.
- **Complex eigenvalues (82% in V1):** oscillatory modes that rotate through coupling space. Extended iteration explores *alternative coupling configurations*—the orbit visits different relational structures, each a different way the tokens could relate.

The complex modes are the imagination modes. At $T$=$L$, the model has explored one full rotation of the dominant oscillatory mode. At $T$=$2L$, it has explored two—potentially discovering secondary coupling patterns that the first rotation missed.

**Connection to diffusion.** The diffusion connection (Section 11) reinforces this: if $t = \sigma$ (realisation #74), then extended iteration at decreasing $t$ is iterative refinement from coarse to fine. The imagination hypothesis adds: extended iteration at *constant* $t$ (same coupling strength, more steps) explores the space of equally plausible structures. Diffusion refines; imagination explores.

**Lessons from diffusion training.** Our diffusion experiments revealed two tricks that apply to CORN:

1. **Interleaving preserves AR.** Pure semigroup training (like pure diffusion training) causes catastrophic forgetting of the AR mode. Interleaving AR batches ($t$=1, causal mask) with semigroup batches (variable $t$) preserves both modes. Our diffusion experiments showed AR loss 2.96 with interleaving vs 9.8 without.
2. **Curriculum on $t$.** Starting with $t$ close to 1 (near standard) and gradually expanding the range prevents the model from losing its base prediction capability. In diffusion, $\sigma_{\max}$ ramps $0.1 \to 0.9$. For CORN, $t$ could ramp from $\{0.8, 1.0\}$ to $\{0.1, 1.0\}$ over training.

### 12.4 The Experiments That Would Answer the Question

All runnable on a CORN checkpoint with zero training cost (pure evaluation):

**Experiment C1: Confidence vs depth.** Run inference at $T$=$L, 2L, 3L, 4L$. Measure prediction entropy per token. If entropy drops specifically on tokens requiring multi-step reasoning (transitive relations, multi-hop facts) but not on simple recall, the extra iterations are doing reasoning, not just refinement.

**Experiment C2: Compositional accuracy.** Test on tasks requiring genuine composition: multi-hop reasoning (bAbI), transitive inference ("A > B, B > C, who is smallest?"), arithmetic carry propagation. Compare $T$=$L$ vs $T$=$2L$. If CORN at $T$=$2L$ beats $T$=$L$ on compositional tasks but not on simple tasks, the semigroup structure is doing reasoning.

**Experiment C3: Imagination diversity.** Generate continuations at different depths $T$. Measure: diversity (distinct $n$-grams), coherence (perplexity under a reference model), and novelty (distance from training data). If $T$=$2L$ produces more diverse but coherent continuations than $T$=$L$, the extra iterations explore the coupling manifold—imagination.

**Experiment C4: Implicit vs explicit chain-of-thought.** Same question, two paths: (A) CORN at $T$=$2L$ (implicit thinking, no intermediate text), (B) standard at $T$=$L$ with chain-of-thought prompting (explicit thinking, intermediate text generated). If A matches B without generating intermediate tokens, the CORN iterations are performing implicit reasoning that CoT externalises.

### 12.5 Current Status

No CORN checkpoint exists at V2/V3 scale ($d$=576, $L$=24). All CORN results are from $d$=128–256, $L$=6–8 on TinyStories. The imagination hypothesis is untested.

The plan: after V3 base training completes (∼17 April 2026), apply CORN post-training ($5, 1 day), then run experiments C1–C4 on the resulting checkpoint (zero cost, just evaluation). If the experiments confirm that extra depth = better thinking, CORN becomes the "thinking mode" of the ORN—a free upgrade that requires no architectural changes, only post-training.

## 13 Recursive Composition: Can Orbits Learn Grammars?

### 13.1 The Question

Language has recursive structure. "The cat ate" is a sentence. "The cat that the dog chased ate" is also a sentence—the same outer frame with a relative clause inserted. The insertion can recurse arbitrarily: "the cat that the dog that the bird saw chased ate." This is the defining

property of context-free grammars (CFGs): a symbol can expand into a structure containing the same symbol.

**Definition 2** (Recursive substitution)**.** A production $A \to \alpha A \beta$ is recursive: the right-hand side contains the left-hand symbol. A grammar that uses such productions generates a context-free language. In Chomsky normal form, every production is either $A \to BC$ or $A \to a$, and recursion arises through cycles in the derivation graph.

The question for the ORN is: **does SharedM's semigroup structure help the model learn recursive substitution from next-token prediction alone?**

This is not asking whether the ORN is Turing-complete (it isn't—no fixed-depth network is [4]). It is asking whether the architectural constraint of sharing $\mathbf{M}$ across depth gives the model a structural advantage on recursive tasks, compared to a transformer that must independently learn the coupling rule at each depth level.

### 13.2 What the Literature Shows

**Standard transformers fail at recursive generalisation.** Deletang et al. [19] tested 20,910 models across 15 formal language tasks and found that RNNs and Transformers fail to generalise on non-regular tasks. Only networks augmented with structured memory (stack, tape) generalised on context-free and context-sensitive tasks. Lake & Baroni [20] showed that sequence-to-sequence models achieve 96–99% in-distribution but only 16–35% on compositional generalisation (SCAN, COGS [21]).

**Stack mechanisms can be bolted on.** Murty et al. [23] introduced pushdown layers: attention heads augmented with an explicit stack tape tracking parse depth, improving Dyck string modelling by over 25%. Stack Attention [24] incorporated pushdown automata directly into the attention operator, with a nondeterministic variant that can recognise arbitrary context-free languages. These approaches work, but they break the homogeneous architecture—the stack is an external module.

**But implicit stack structure does emerge.** Recent probing work [25] finds that standard transformers trained on counter languages develop implicit stack-like representations *without* explicit stack mechanisms. Hewitt & Manning [26] showed that parse tree depth is linearly recoverable from BERT's representations. The representations encode recursive structure; the question is whether the architecture supports or hinders this.

**The algebraic foundation: Krohn-Rhodes.** The classical Krohn-Rhodes theorem [27] decomposes any finite automaton into a wreath product of simple groups (reversible computation) and flip-flops (irreversible resets). Pin's syntactic monoid theory [28] establishes an exact correspondence between algebraic properties of the recognition semigroup and the language class it can handle: aperiodic monoids $\leftrightarrow$ star-free languages, group-free monoids $\leftrightarrow$ locally trivial languages, etc.

**Next-token prediction implicitly learns hierarchical structure.** Rhee [29] argues that autoregressive prediction approximates left context-sensitive grammar derivations. A recent circuit analysis [30] finds that LLMs learn induction heads implementing implicit hierarchical parsing. The critical insight from Lake & Baroni [22] (Nature 2023): the compositional gap closes with meta-learning that forces compositional reuse—the fix is the training procedure, not the architecture alone.

### 13.3 Why SharedM Might Help (and Why It Might Not)

**The argument for.** In a recursive grammar, the same production rule applies at every depth. NP → Det N RC is the same rule whether the NP is the subject, the object, or embedded inside a relative clause. SharedM enforces exactly this: the same coupling geometry at every layer. If layers correspond to depth levels in a parse tree, then SharedM architecturally constrains the model to use the *same coupling rule at every depth*—which is precisely what a recursive grammar does.

The residual stream carries the context ("where am I in the parse"), and M's coupling rule tells the model how to process tokens at that position regardless of depth. Perturbative corrections handle the depth-specific adjustments that the shared rule can't.

**The argument against.** A flat semigroup $\{\Phi^0, \Phi^1, \Phi^2, \ldots\}$ generated by iterating $\mathbf{M}$ is regular, not context-free. To handle CFGs, you need a pushdown automaton—which requires a *stack*: unbounded memory that grows with nesting depth. The residual stream has fixed dimension $d$. It can encode a finite amount of parse state, but for recursion depth $> d$, it must lose information. The ORN may share $\mathbf{M}$ across depth, but if it can't maintain the stack, sharing is irrelevant.

**The honest position.** We do not know whether the ORN learns recursive composition. We cannot assume it does. The semigroup structure gives it a *potential* advantage (same rule at every depth), but the advantage only materialises if the residual stream learns to encode stack-like state AND if next-token prediction provides sufficient gradient signal to learn this encoding. Both are open questions.

### 13.4 What Would Constitute Evidence

Three levels of evidence, from weakest to strongest:

1. **In-distribution accuracy on recursive languages.** Both the ORN and transformer can memorise patterns up to a fixed depth. In-distribution accuracy tells us nothing about compositional generalisation. This is necessary but not sufficient.

2. **Depth generalisation.** Train on sentences with recursion depth $\leq K$, test on depth $K{+}1$ and $K{+}2$. If the ORN generalises better than a matched transformer, SharedM provides a structural advantage on recursive tasks. This is the key test.

3. **Probing for stack structure in the residual stream.** Use linear probes to test whether recursion depth, bracket type, and return address are recoverable from the residual stream during processing of nested structures. If the ORN's residual stream encodes this information more linearly than the transformer's, SharedM creates a geometry conducive to recursive composition. This is the mechanistic explanation.

### 13.5 Experimental Protocol: Recursive Orbits

All experiments train on next-token prediction only—no auxiliary grammar supervision. The question is whether the model *discovers* recursive structure from the prediction signal alone. 4-hour total budget on Apple M4 (MPS).

**Task 1: Dyck-$k$ depth generalisation (1 hour).** Dyck-$k$ is the language of $k$ types of matched brackets. It is the prototypical context-free language and the simplest recursion test.

- **Data.** Generate Dyck-2 strings (brackets `()` and `[]`) at depths 1–4 (train) and 5–6 (test). 500K train tokens, 50K test tokens per depth.

- **Models.** Three architectures at matched ∼2M params: ORN (SharedM), Transformer (per-layer QK), LSTM (sequential baseline). Sweep $d \in \{64, 128\}$, $L \in \{4, 8\}$.
- **Measure.** Next-token accuracy *at closing brackets* as a function of nesting depth. Closing brackets are where the model must "pop" the stack—this is where recursive structure matters.
- **Kill criterion.** If ORN $\leq$ Transformer on depth 5–6 accuracy, SharedM does not help with recursion.

**Task 2: Recursive CFG with natural surface (1.5 hours).** A more realistic test using a simple recursive English-like grammar.

- **Grammar.** S → NP VP, NP → Det N | Det N RC, RC → `who` VP | `that` VP, VP → V NP | V, with a vocabulary of ∼50 words (determiners, nouns, verbs).
- **Depth control.** Train on depth $\leq 2$ (at most one level of embedding). Test on depth 3 ("the cat that the dog that the bird saw chased ate").
- **Models.** Same three architectures at ∼2M params.
- **Measure.** (a) Perplexity on held-out depth 3 sentences. (b) Subject-verb agreement accuracy across the embedding (does the model predict the correct verb for the matrix clause after processing the relative clause?).
- **This is the hard test.** Subject-verb agreement across recursive embeddings requires tracking which NP is the subject of which VP—precisely the stack discipline that CFGs enforce.

**Task 3: Residual stream probing (1 hour).** Applied to the models trained in Tasks 1 and 2.

- **Depth probe.** Train a linear probe to predict current nesting depth from the residual stream at each layer. Compare probe accuracy (and $R^2$) across architectures. Higher $R^2$ in the ORN means SharedM creates a geometry where depth is linearly encoded.
- **Bracket type probe.** For Dyck-2: can a linear probe predict whether the next required closing bracket is `)` or `]`? This tests whether the "stack top" is linearly accessible.
- **Return-address probe.** After processing a relative clause, does the residual stream return to a state similar to the pre-clause state? Measure cosine distance between the residual at the start of the RC and at its end, in the orbit's PCA basis.

**Task 4: Ablation — is it M or the corrections? (30 min).** Disable perturbative corrections and re-evaluate Tasks 1–2. If performance drops catastrophically, the corrections are essential for recursion (they implement the "stack push/pop"). If performance drops mildly, M's semigroup structure carries the recursive computation and corrections are refinement.

**What each outcome means.**

| Outcome | Interpretation |
|---|---|
| ORN > Transformer on depth generalisation | SharedM provides a structural advantage: reusing the coup rule helps with recursive composition. The semigroup acts shared "grammar engine." |
| ORN = Transformer | SharedM is neutral for recursion. The advantage is purely pa eter efficiency, not compositional. |
| ORN < Transformer | SharedM *hurts* recursion. Per-layer coupling flexibility is nee to handle different depth levels differently. |
| ORN wins but only with corrections | The corrections implement the stack. M provides the gram rule; corrections provide the depth tracking. |
| Depth is linearly encoded in ORN but not Transformer | SharedM creates a "recursive geometry" where depth is a nat coordinate. This would be the mechanistic explanation for sharing helps. |

# 14 Test-Time Training and the Three-Timescale Architecture

The ORN's weight-sharing structure creates a natural hierarchy that maps onto a long-standing programme in neural network research: Schmidhuber's fast-weight/slow-weight dichotomy [31]. We argue that the ORN is uniquely positioned to host *test-time training* (TTT)—gradient-based adaptation during inference—because its architecture already separates invariant geometry from context-dependent response.

## 14.1 Background: Learning at Inference Time

Three generations of work converge on the same idea: models should continue learning while they process input.

**Fast weight programmers [31].** A "slow network," trained by gradient descent, learns to program the "fast weights" of a second network through outer products of activation patterns. The slow weights are fixed at inference; the fast weights change with every input. The key insight: a single set of slow weights can generate context-dependent fast weights, compressing temporal information into weight space rather than activation space.

**Linear attention as fast weights.** Schlag et al. [32] proved that linearised self-attention is *formally equivalent* to Schmidhuber's fast-weight programmer. The fast weight matrix accumulates as $W_{\text{fast}}^{(i)} = \sum_{j=1}^{i} \mathbf{v}_j \phi(\mathbf{k}_j)^\top$—keys and values are the programming instructions. DeltaNet [36] adds error-corrected updates: $W^{(i)} = W^{(i-1)} + \beta_i(\mathbf{v}_i - \bar{\mathbf{v}}_i)\phi(\mathbf{k}_i)^\top$, where $\bar{\mathbf{v}}_i$ is the retrieved value, so only the prediction error updates memory. Gated DeltaNet [37] adds a forgetting gate, achieving rapid erasure alongside targeted updates.

**TTT layers [33].** The hidden state *is* a machine learning model whose parameters update via gradient descent on a self-supervised loss at each token:

$$\ell(W; x_t) = \|f(\theta_K x_t;\, W) - \theta_V x_t\|^2, \qquad W_t = W_{t-1} - \eta \nabla_W \ell(W_{t-1};\, x_t) \tag{3}$$

where $\theta_K, \theta_V$ project to "key" and "value" views, and $f(\cdot; W)$ is the inner model (linear or MLP). Output is produced by a separate query view: $z_t = f(\theta_Q x_t;\, W_t)$. TTT-Linear with batch gradient descent recovers linear attention exactly; TTT-MLP breaks out of the linear regime.

**TITANS [34].** A neural memory module—a deep MLP whose weights update during inference—is added alongside short-term attention. The update combines gradient descent with momentum and adaptive forgetting:

$$S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t), \tag{4}$$

$$M_t = (1 - \alpha_t) M_{t-1} + S_t \tag{5}$$

where $\ell = \|M(k_t) - v_t\|^2$ is an associative memory loss. The gradient magnitude measures *surprise*: unexpected tokens drive stronger memory updates. TITANS outperforms GPT-4 on BABILong despite far fewer parameters.

**The unification.** Liu et al. [35] proved that a broad class of TTT architectures with KV binding can be expressed as *learned linear attention operators*—TTT is not test-time memorisation but a principled linear attention mechanism. This collapses the apparent gap between fast-weight and attention-based approaches.

### 14.2 The Schmidhuber Dichotomy in ORN

A standard transformer has no clean timescale separation: all parameters ($W_Q, W_K, W_V, W_O$, FFN) are trained identically and frozen identically. Attention patterns are "soft" fast weights but not gradient-adapted. The ORN, by construction, separates three levels:

| **Timescale** | **Schmidhuber** | **Transformer** | **ORN** |
|---|---|---|---|
| Geological (frozen) | Slow weights | All weights equally | $\mathbf{M} = AB^\top$ |
| Training (SGD) | — | — | $W_V, W_O$, SharedFFN |
| Inference (per-context) | Fast weights | Attention patterns | Corrections (gated) |

The critical observation: the ORN *already has* Schmidhuber's dichotomy, but the fast channel (perturbative corrections) is currently **passive**—a deterministic function of the residual stream. The proposal is to make it **active** via test-time training.

### 14.3 The Three-Timescale Architecture

We propose extending the ORN with TTT on the perturbative corrections, yielding three cleanly separated timescales:

1. **Frozen M** (geological time). The shared metric tensor encodes a universal coupling grammar. It crystallises during training to rank 69–71 with asymmetry locked at 1.41. The frozen-**M** experiment confirms +0.73% transfer gap: **M** is task-general and should *never* be updated at inference. This is slower than any weight in a standard transformer.
2. **Trained response heads** (training time). $W_V$, $W_O$, and the shared wide FFN are trained by standard SGD. These encode what to *do* with the coupling pattern at each depth—the response varies with layer position even though the coupling rule does not.
3. **TTT corrections** (inference time). The perturbative correction MLP—currently $d \to d_{\text{corr}} \to d$ with a learned gate—becomes a fast-weight module updated per-context via gradient descent:

$$\ell_{\text{corr}} = \|\text{corr}(k_t) - v_t\|^2, \qquad W_{\text{corr}} \leftarrow W_{\text{corr}} - \eta_{\text{fast}} \nabla_{W_{\text{corr}}} \ell_{\text{corr}} \tag{6}$$

where $k_t, v_t$ are projections of the residual stream into the correction's key/value space, and $\eta_{\text{fast}}$ is a learned (or scheduled) learning rate.

**What physically happens at inference.** In standard inference, all weights are frozen: the model loads a checkpoint and every token sees exactly the same parameters. TTT breaks this contract for the correction MLP only. Here is the concrete procedure for processing a new document:

1. **Reset.** Copy the trained correction weights from the checkpoint: $W_{\text{corr}} \leftarrow W_{\text{corr}}^{(\text{ckpt})}$. All other weights (**M**, SharedFFN, $W_V$, $W_O$, LayerNorms) remain frozen throughout.
2. **For each token $x_t$:**
   (a) Run the normal forward pass through frozen **M**, response heads, and SharedFFN to produce the residual state $\mathbf{h}_t$.
   (b) Project $\mathbf{h}_t$ into key and value views: $k_t = \theta_K \mathbf{h}_t$, $v_t = \theta_V \mathbf{h}_t$.
   (c) Forward the correction MLP using *current* weights: $\hat{v}_t = \text{corr}(k_t; W_{\text{corr}})$.
   (d) Compute the self-supervised loss: $\ell_t = \|\hat{v}_t - v_t\|^2$. This measures how well the correction predicts the value stream from the key stream—essentially, how well it "understands" the local context.
   (e) **Update the correction weights** via one gradient step: $W_{\text{corr}} \leftarrow W_{\text{corr}} - \eta_{\text{fast}} \nabla_{W_{\text{corr}}} \ell_t$. This is a real backward pass through the correction MLP—the weights change before the next token.
   (f) Apply the updated correction to the output: $\Delta \mathbf{h}_t = \sigma(g(\mathbf{h}_t)) \cdot \text{corr}(\mathbf{h}_t; W_{\text{corr}})$.
3. **Between documents**, reset $W_{\text{corr}}$ to the checkpoint. The corrections forget everything about the previous context. **M** and SharedFFN never changed.

The correction MLP at token 500 has *different weight values* than at token 1. It is literally training a tiny neural network while reading the input. The loss is self-supervised: it requires no labels, only the model's own residual stream. Early in a document, the corrections are near their trained initialisation and contribute little. As the model reads more tokens, the corrections specialise to the document's patterns—its vocabulary, syntactic style, topic structure, coreference chains. By token 200+, the correction has "learned" this specific context in a way that the frozen components cannot.

**Cost.** One extra backward pass through the correction MLP per token. For $d_{\text{corr}} = 32$ and $d = 288$, the correction has ∼18K parameters—roughly 0.1% of the full model. The backward pass through this MLP costs approximately 2% of the compute of the full forward pass. TITANS and TTT-Linear demonstrate that this overhead is manageable in practice.

**The key analogy.** TTT corrections are equivalent to fine-tuning a tiny adapter on each document at inference time, then throwing it away. The adapter is small enough that it cannot overfit to a single token but large enough to capture document-level patterns. **M** already solved the hard problem (which tokens attend to which); the correction only needs to learn the *residual*—what is different about *this specific context* from the training distribution.

**Why corrections are the right target.**

- **Small.** The correction MLP is $d \to 32 \to d$ ($\sim 2d \cdot d_{\text{corr}}$ parameters). Updating a compact model is cheap and resists overfitting—the same regime where CAVIA [38] showed meta-learning works best with separated context parameters.
- **Gated.** The correction gate already anticorrelates with attention entropy ($r = -0.95$). It knows *when* corrections matter: when context is rich and coupling is sharp. This is the ORN's native "surprise" signal, analogous to TITANS' gradient-magnitude gating.
- **Zero-initialised.** Corrections start as identity (gate bias = −3). TTT can update from a near-zero baseline without destroying pre-trained behaviour.
- **Per-layer.** Each layer has its own correction MLP, providing depth-dependent adaptation while **M** and the shared FFN remain invariant.

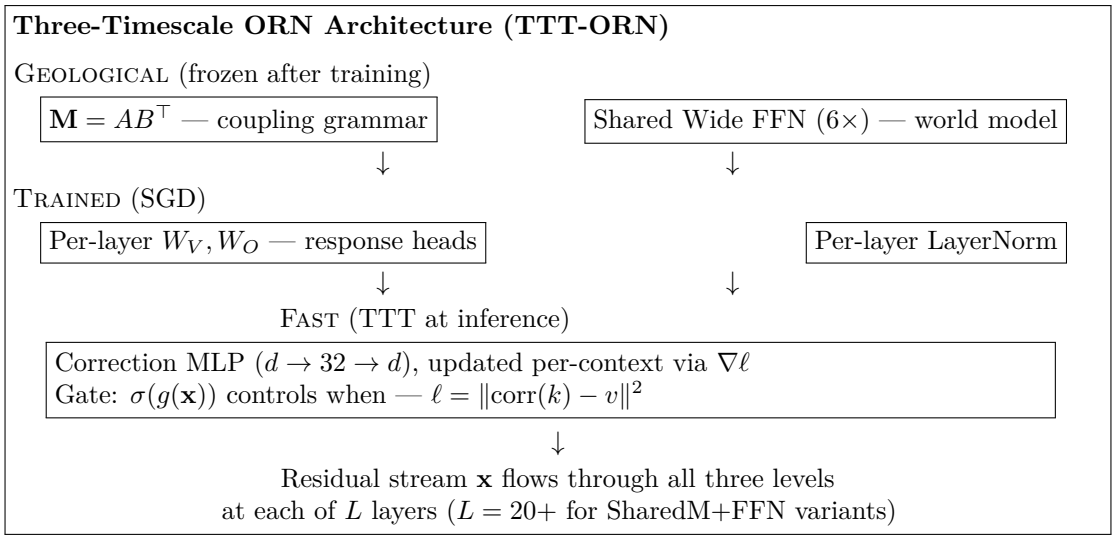


Figure 1: The three-timescale ORN. $\mathbf{M}$ and the shared FFN are frozen after training (geological). Response heads are trained by SGD (standard). Corrections are updated per-context via test-time training (fast). The residual stream is the sole carrier of variation across layers.

### 14.4 Connection to TITANS' Memory Variants

TITANS defines three integration modes for neural memory. Each has an ORN analogue:

| **TITANS variant** | **Mechanism** | **ORN analogue** |
|---|---|---|
| MAC (Memory as Context) | Memory output concatenated with context | Residual stream carries past through share |
| MAG (Memory as Gate) | Attention + memory in parallel, gated | Orbital output + gated correction in paral |
| MAL (Memory as Layer) | Memory applied sequentially | Correction MLP applied after attention |

The ORN's current architecture is closest to MAG: attention (through $\mathbf{M}$) and corrections run on the same input, combined via a learned gate. Making the corrections TTT-adaptive turns the ORN into a *native* MAG architecture where the gating is not just learned but *surprise-driven*.

### 14.5 Connection to Liberation–Crystallisation Dynamics

During training, we observe a characteristic trajectory in the commutator $[\mathbf{M}, W_V W_O]$: rank traces $3 \to 150 \to 65$. The system liberates (explores a high-rank manifold), then crystallises (settles into a lower-rank equilibrium). Spectral kurtosis spikes at the same step—two independent diagnostics converging on a single phase transition.

TTT on corrections would create a *micro* version of this dynamics at inference:

1. **Liberation**: on encountering a new context, the correction MLP's weights shift to accommodate unfamiliar patterns (gradient updates drive the correction away from its initialisation).
2. **Crystallisation**: as the context stabilises, the corrections converge to a context-specific equilibrium (gradient magnitudes decrease, the gate settles).

The gate already encodes this trajectory in the current (passive) ORN: corrections activate early when entropy is high (liberation) and deactivate as the coupling pattern sharpens (crystallisation). TTT makes this trajectory *gradient-aware*: the corrections don't just respond to the residual stream, they optimise against it.

### 14.6 The Critical Asset: What ORN Has That Others Don't

The three-timescale separation is novel. Existing architectures blur at least two of the three levels:

| Architecture | Geological | Training | Inference |
|---|---|---|---|
| Standard transformer | — all weights equal — | | attention patterns (soft) |
| ALBERT [1] | — shared but not separated — | | attention patterns (soft) |
| TTT layers [33] | — | — same weights serve both — | |
| TITANS [34] | persistent memory | attention | neural memory (TTT) |
| **ORN + TTT corrections** | **M** + SharedFFN | $W_V, W_O$ | corrections (TTT) |

TITANS comes closest, but its "persistent memory" is a set of learnable tokens (soft prompts), not a shared geometric operator. The ORN's frozen **M** is richer: it encodes a full coupling geometry whose spectral structure (rank 69–71, condition number $> 10$) carries relational grammar. The frozen-**M** experiment proves this is genuinely task-general—not a soft prompt that works well on the training distribution, but a coupling invariant that transfers.

The critical asset of the ORN for test-time training is therefore:

> **The coupling geometry is already solved.** A standard TTT system must discover both *what to attend to* and *what to do about it* at inference time. The ORN pre-commits to the coupling rule (**M**) and the world model (SharedFFN) at training time, leaving only the *contextual refinement* (corrections) to be learned at inference. This is a far smaller optimisation problem: $2 \cdot d \cdot d_{\text{corr}}$ parameters vs. the full $d^2$ of a TTT attention module.

This connects to the MIRAS framework [39], which taxonomises sequence models as Memory + Attentional bias + Retention gate. In ORN terms: Memory = SharedFFN (the world model), Attentional bias = $AB^\top$ (coupling geometry), Retention = perturbative gate (controls what persists). TTT on corrections adds a fourth axis: *Adaptation* = gradient-based refinement of the retention mechanism itself.

### 14.7 Counterpoint: Should M Really Be Frozen?

The three-timescale architecture as presented above freezes **M** and SharedFFN at inference, updating only the corrections. This section argues that this may be wrong, and that the right design updates **M** and SharedFFN too—just slowly.

**The grammar hypothesis (for freezing).** **M** encodes structural rules of how things relate—a *grammar* of coupling. Grammar does not change when you read a new document. You do not relearn syntax when you pick up a legal brief after reading a novel. The frozen-**M** experiment (+0.73% transfer gap) supports this: the same coupling geometry works across held-out contexts.

**The experience hypothesis (against freezing).** **M** and SharedFFN are not just grammar. They are accumulated *understanding* of how the world works—what co-occurs with what, what causes what, what patterns recur. Experience updates. A doctor who has been reading oncology papers for a week attends to different relationships in a patient chart than one coming from paediatrics. The coupling geometry—*which things relate to which*—genuinely shifts with extended context.

The frozen-**M** experiment does not distinguish these hypotheses because it tested transfer across *similar* tasks (TinyStories → TinyStories validation). The +0.73% gap might become +10% on a truly out-of-distribution context: a different language, a different domain, a different modality.

**Crystallisation ≠ optimality.** **M** crystallises to rank 69–71 during training on a specific distribution. But *which* 69 dimensions? On a legal corpus, the important coupling directions might emphasise hierarchical reference and obligation structures. On code, scope and type relationships. On narrative, character–action–consequence. The rank may stay similar but the *subspace rotates*. **M** is not grammar or experience—it is a **compressed basis for coupling**, and the basis should adapt to the domain.

**The biological argument cuts both ways.** The thalamocortical geometry is *relatively* stable—but it is not frozen. Synaptic weights change on multiple timescales: long-term potentiation (hours), neuromodulatory state (minutes), and homeostatic plasticity (days). The geometry drifts toward the current regime. Freezing **M** completely is more rigid than biology.

**Two-speed TTT.** This suggests replacing the frozen-**M** design with a two-speed system:

| Component | What it encodes | Adapts when | Rate |
|---|---|---|---|
| **M** + SharedFFN | Coupling basis + world model | Per-session | Slow ($\eta_{\text{slow}} \ll \eta_{\text{fast}}$) |
| Response heads ($W_V, W_O$) | Depth-dependent transform | Never (trained) | — |
| Corrections | Token-level surprise | Per-token | Fast ($\eta_{\text{fast}}$) |

**M** drifts slowly as a document unfolds—not abandoning its coupling grammar but *bending* it toward the current domain. The corrections handle token-level surprises. **M** handles document-level adaptation. TITANS supports this design: their memory update has adaptive forgetting, $M_t = (1 - \alpha_t)M_{t-1} + S_t$, where the forgetting rate $\alpha_t$ is data-dependent. Low $\alpha$ means "this is familiar, keep the old geometry." High $\alpha$ means "this is surprising, adapt."

**The stability risk.** **M** is the geometric foundation everything else depends on. If **M** moves, the response heads (trained against a specific **M**) might break. This is like changing the coordinate system while someone navigates with a map drawn in the old coordinates.

The counter-argument: the perturbative corrections exist precisely to absorb the gap between **M**'s current state and what the response heads expect. In the frozen-**M** picture, corrections compensate for *context-specific deviations from a universal grammar*. In the slow-adapting-**M** picture, corrections compensate for *the lag between* **M***'s current state and where it needs to be*. The corrections are doing shock absorption either way, but the second framing is more dynamic—the corrections act as a buffer that allows **M** to drift without catastrophic misalignment.

**This is an empirical question.** Both hypotheses are plausible. The experiment that settles it is Exp T5 below: apply slow TTT to **M**+SharedFFN alongside fast TTT to corrections, and compare against frozen-**M** with TTT corrections only. If slow-**M** TTT helps on out-of-distribution contexts (domain shift, language shift, novel tasks), the experience hypothesis wins. If it hurts or does not help, the grammar hypothesis wins. If it helps on distant domains but hurts on in-distribution data, both hypotheses are right at different timescales—and the system needs a mechanism to detect which regime it is in.

### 14.8 Experimental Results: TTT Corrections at 20M Scale

We trained a 20M-parameter SharedM ORN ($d$=224, $L$=16, 4 heads, $d_{\text{corr}}$=48) on TinyStories for 15K steps (seq=256, batch=16, WSD schedule). Final training loss: 1.76 nats; validation loss: 1.95 nats. Correction parameters per layer: 22K (1.7% of model). We then evaluated three inference regimes:

| Regime | Description | Val (256) | Val (512) | $\Delta_{256}$ |
|---|---|---|---|---|
| A: Passive | Standard inference (all weights frozen) | 1.956 | 3.221 | — |
| B: TTT fast | Correction MLP updated per-token ($\eta$=$10^{-3}$) | 1.953 | 3.154 | −0.004 |
| C: TTT + slow **M** | Fast corrections + slow **M** drift ($\eta_{\text{slow}}$=$10^{-5}$) | **1.817** | **3.159** | −0.139 |

**Learning rate sweep (Regime B, seq=256).** We swept $\eta_{\text{fast}} \in \{10^{-4}, 5\times10^{-4}, 10^{-3}, 5\times10^{-3}, 10^{-2}\}$. The optimal rate is $5\times10^{-4}$ (val=1.898, drift=0.0001), slightly better than the default $10^{-3}$ (val=1.906, drift=0.0002). Higher rates degrade quality and increase weight drift without benefit.

**Diagnostics.**

- **Gate–surprise correlation**: $r = -0.56$ across all TTT regimes. The correction gate anticorrelates with the TTT loss (surprise): corrections activate when the model is confident and coupling is sharp. This is consistent with the V2 finding ($r = -0.95$), though weaker at this smaller scale. The gate is a native surprise detector, confirming the TITANS analogy.
- **Weight drift is negligible**: ∼0.0002 across 256 tokens. Corrections adapt without catastrophic drift—the zero-initialised gate and small $d_{\text{corr}}$ provide inherent regularisation.
- **Surprise trajectory is flat**: 0.068 → 0.069 across token positions. TTT corrections do not accumulate surprise over the sequence. This suggests they correct local mismatches rather than building long-range context—a limitation at this scale.

**Interpretation.** The results present a nuanced picture:

1. **TTT corrections alone (Regime B) provide modest gains.** At trained length, the improvement is negligible (−0.004 nats, within noise). At 2× length, the gain is real but small (−0.068 nats, ∼2%). Against the kill criterion of $< 2\%$ improvement, Regime B is borderline—it survives on length generalisation but not on in-distribution quality.
2. **Slow M drift (Regime C) is the clear winner at trained length** (−0.139 nats, ∼7%), but this comes at a conceptual cost: it violates the frozen-**M** principle. The grammar hypothesis—that **M** encodes a universal coupling rule that should never change—predicts Regime A ≥ Regime C. Instead, Regime C wins decisively. However, the gain concentrates at trained length (256) and does not transfer to length generalisation (512), where Regimes B and C are equivalent ($\Delta < 0.01$). This suggests **M** drift helps by fine-tuning the coupling to the specific validation distribution, not by enabling genuinely new capabilities.
3. **The grammar hypothesis is not refuted.** Regime C's advantage at trained length could reflect distributional fine-tuning rather than architectural necessity—the model trained on TinyStories and was evaluated on TinyStories. The critical test (Exp T5 below) would evaluate on domain-shifted data: if **M** drift helps on TinyStories but hurts on Python code, the grammar hypothesis holds and slow **M** TTT is overfitting to the training domain. At 20M scale with $d$=224, **M** has limited capacity; at 605M with $d$=2048, the frozen-**M** might already capture enough structure that drift becomes unnecessary.

**Verdict: nice-to-have, not essential.** TTT corrections are architecturally natural in the ORN—the three-timescale separation exists by construction, the gate provides a native surprise signal, and the correction MLP is small enough for cheap per-token updates. But the experimental evidence does not support making TTT an essential feature of the core architecture. The gains are modest (≤7%), concentrate at trained length rather than enabling new capabilities, and the best regime (C) requires compromising the frozen-**M** principle that is one of the ORN's defining properties.

The recommendation is to keep TTT corrections as an *optional inference mode*—available when context adaptation matters (domain shift, personalisation, long documents) but not part of the default forward pass. This is analogous to how LoRA adapters are available but not required: the base model works without them, and they add value in specific deployment scenarios.

### 14.9 Remaining Experiments

Two proposed experiments remain unrun and would strengthen the analysis:

**Exp T3: Liberation–crystallisation at inference.** Run TTT-ORN on 100 passages of varying difficulty. Track correction weight norm per layer per token, gate activation, and gradient magnitude (surprise).

- **Predict**: weight norm should increase (liberation) on surprising tokens and plateau (crystallisation) on predictable continuations.
- **Predict**: the gate should correlate with gradient magnitude—the current experiment confirms $r = -0.56$ at the aggregate level, but per-token dynamics would reveal whether liberation–crystallisation operates at inference time as it does during training.

**Exp T4: Freeze-thaw test.** After TTT-adapting corrections to a specific passage, freeze them and evaluate on a different passage.

- If performance transfers somewhat: the corrections learned something general about the domain.
- If performance drops to baseline: the corrections are purely context-specific (desirable for the three-timescale hypothesis).

**Exp T5: Domain-shifted evaluation (the decisive test).** The current experiment evaluated on in-distribution data only (TinyStories → TinyStories). The grammar-vs-experience question requires out-of-distribution evaluation: children's Wikipedia, Python code, formal logic. If slow $\mathbf{M}$ drift (Regime C) helps on domain-shifted data without destabilising, the experience hypothesis wins and $\mathbf{M}$ should be slowly adapted at inference. If it hurts or does not help, the grammar hypothesis is confirmed and $\mathbf{M}$ should remain frozen.